Natural Language Processing

Episode 8

Model Compression & Acceleration

by Andrei Panferov

Chapter 1: Why Should You Care?

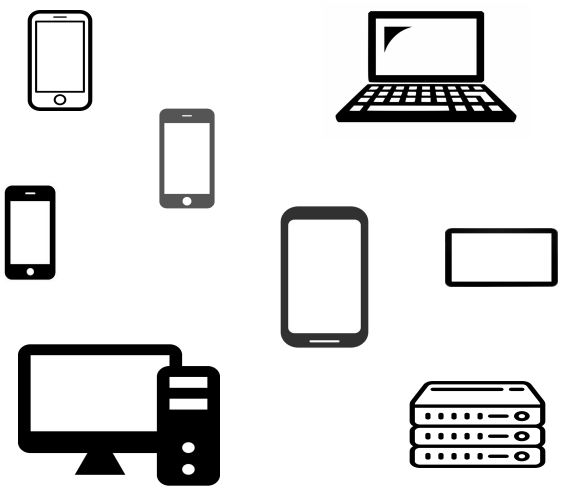
System Requirements

Model Size
(mega)bytes

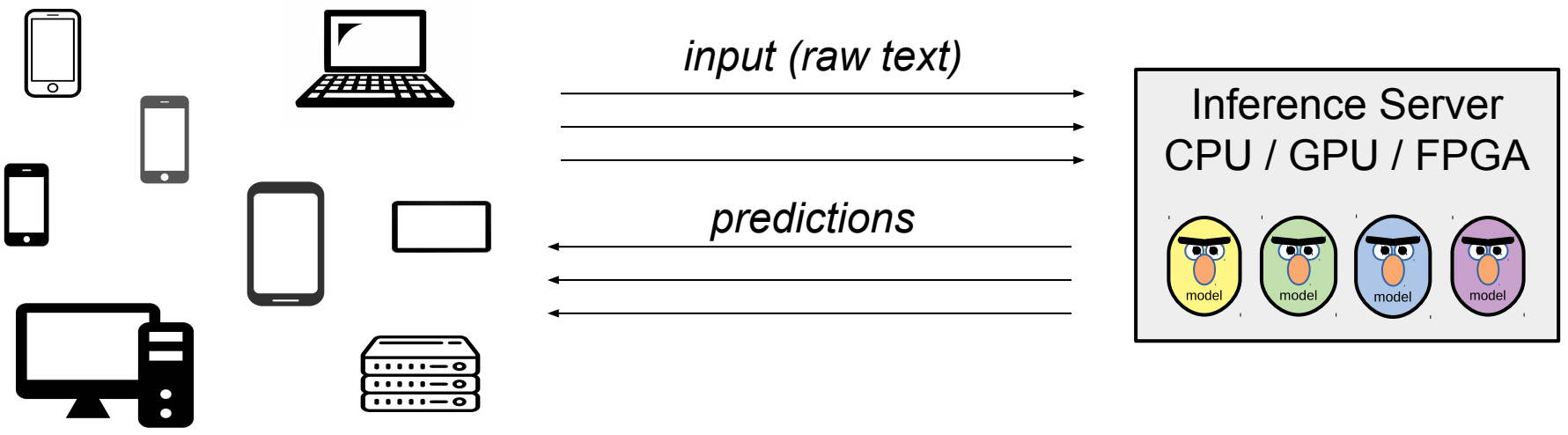
Throughput
samples/second

Latency
ms@percentile

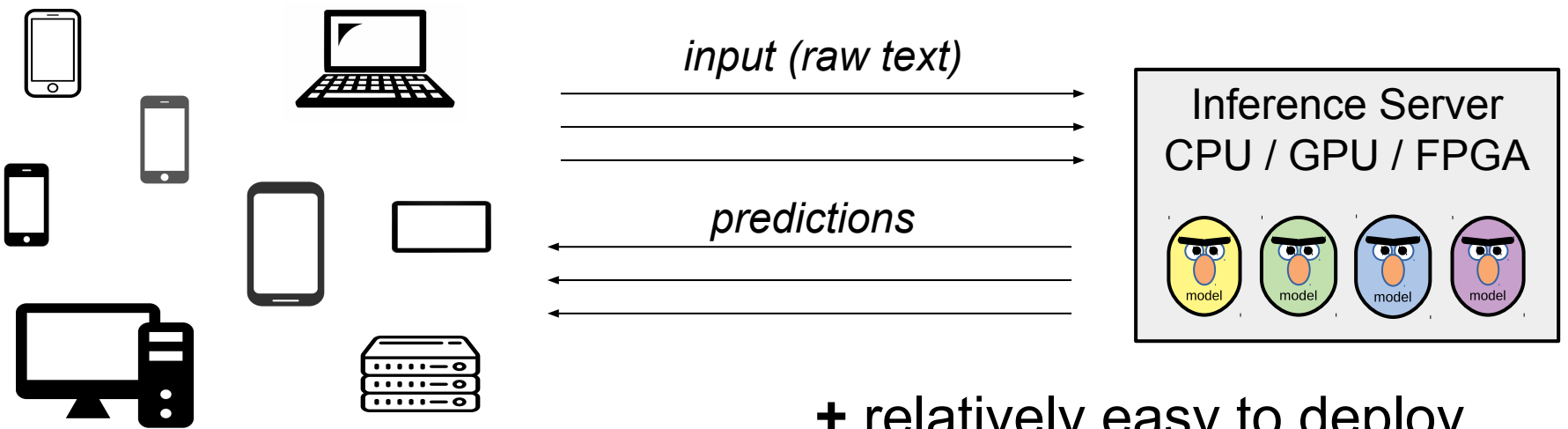
Inference Server



Inference Server

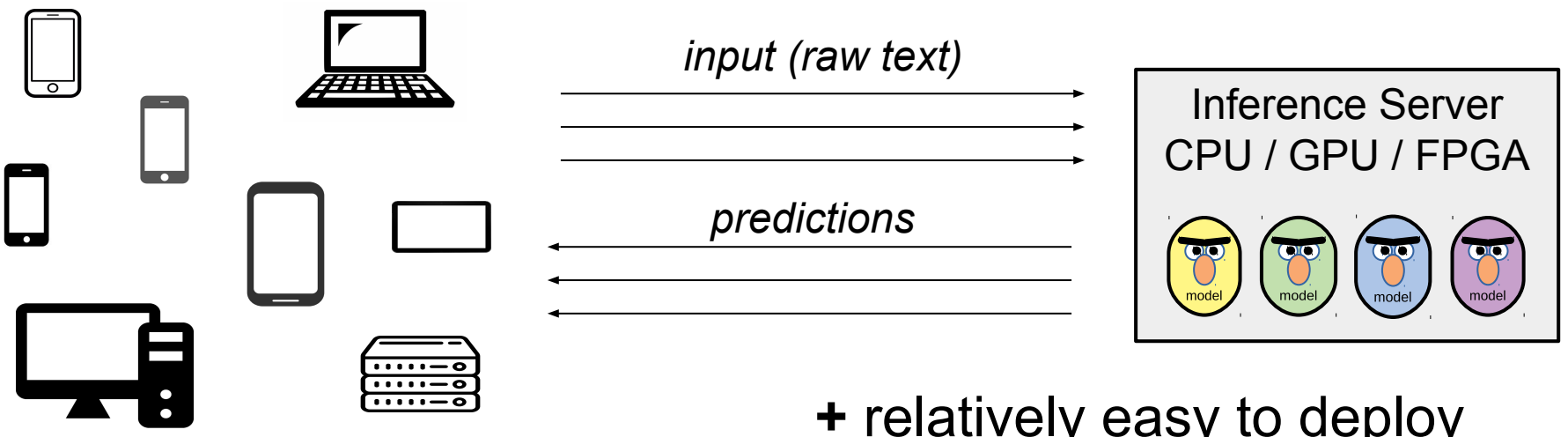


Inference Server



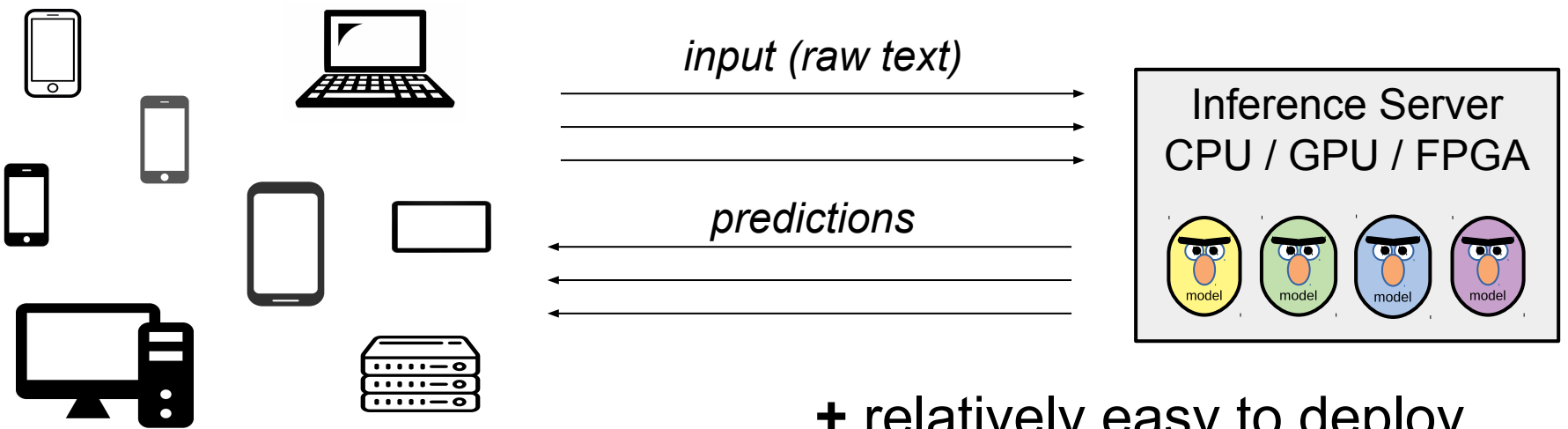
+ relatively easy to deploy
+ you control model & inference
+ clients don’t run compute

Inference Server



+ relatively easy to deploy
+ you control model & inference
+ clients don’t run compute

Inference Server

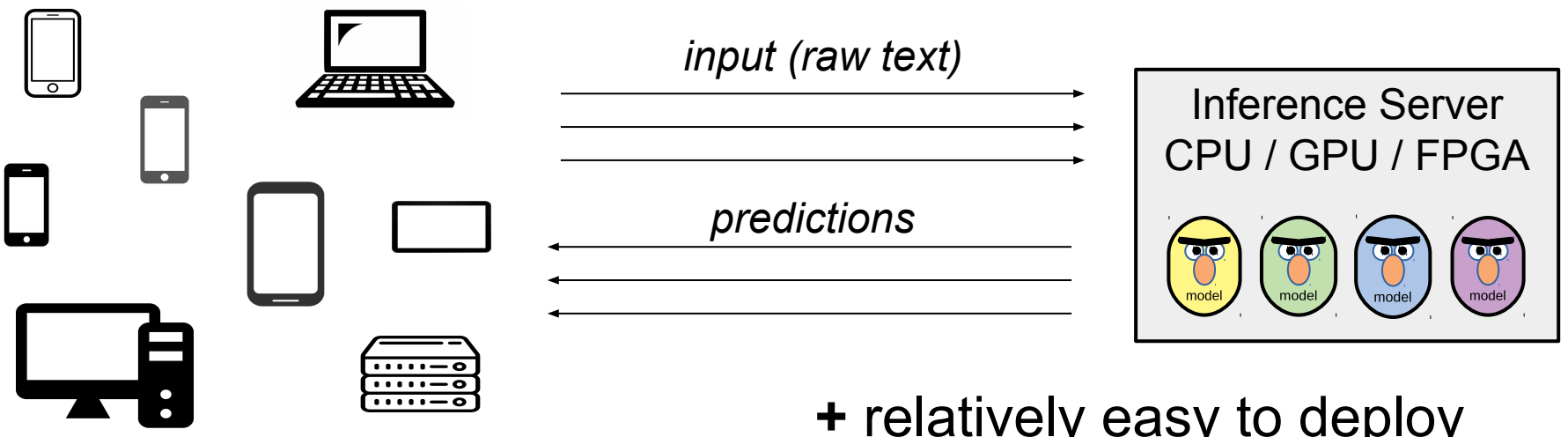


Which is the most important?

? ? ?

+ relatively easy to deploy
+ you control model & inference
+ clients don’t run compute

Inference Server



+ relatively easy to deploy
+ you control model & inference
+ clients don't run compute

Priorities:

Inference Server

- Group inputs into batches (e.g. by length)
  - *improves throughput at the cost of latency*
- Multiple servers with load balancing

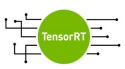
Inference Server

- Group inputs into batches (e.g. by length)
  - *improves throughput at the cost of latency*
- Multiple servers with load balancing

  improves throughput at the cost of your budget :)

Popular frameworks:


vLLM



Triton Inference Server


Custom model-dependent code

priorities

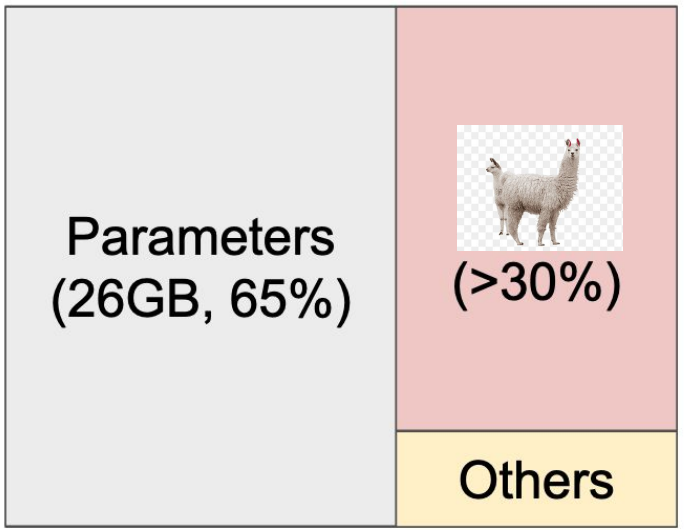
efficiency $\ll$ developer time

efficiency $\approx$ developer time

efficiency $\gg$ developer time

What?

Question: what did behind the Llama?



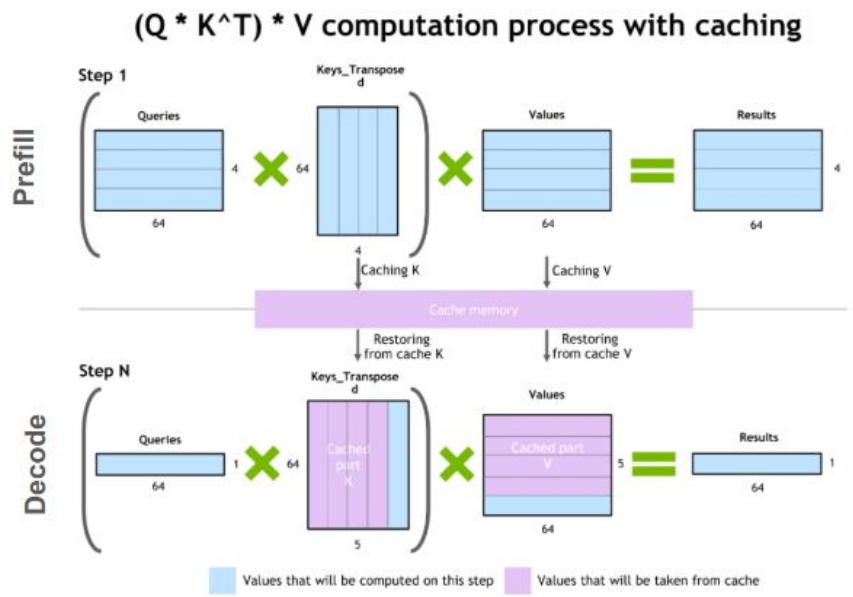
NVIDIA A100 40GB

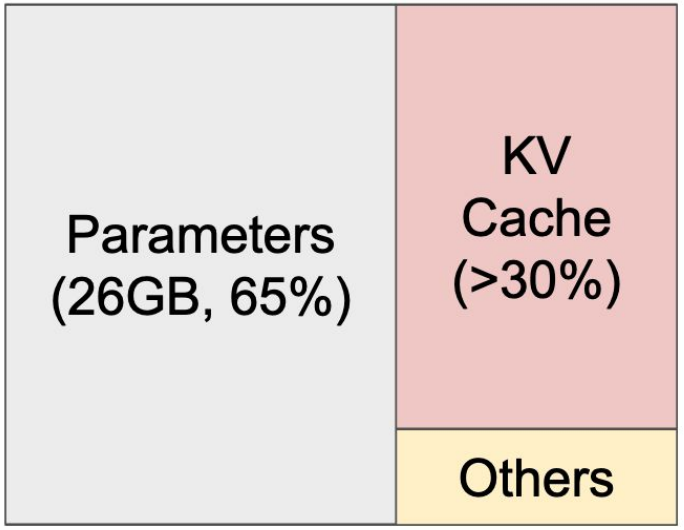
Image: arxiv.org/pdf/2309.06180

KV-Caches

Question: what did behind the Llama?

Answer: KV Cache



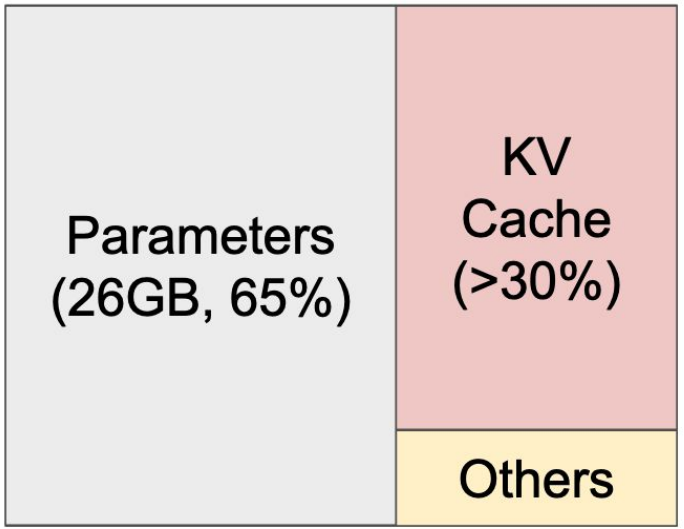


NVIDIA A100 40GB

Image: arxiv.org/pdf/2309.06180

KV-Caches

Question: how many parallel requests we can serve if we have 20Gb of extra DRAM and one token takes up 200kb of cache?
Answer: it depends…



NVIDIA A100 40GB

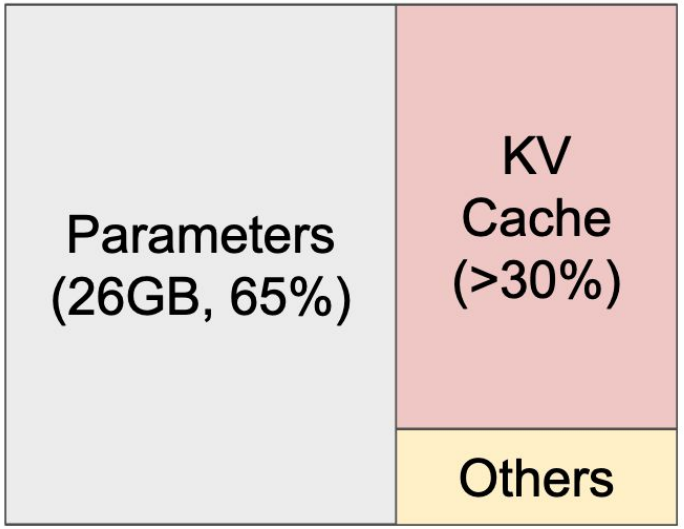
Image: arxiv.org/pdf/2309.06180

KV-Caches

Question: how many parallel requests we can serve if we have 20Gb of extra DRAM and one token takes up 200kb of cache?
Answer: it depends…

What if requests were very diverse:

- 2000 prompt tok, 1 gen tok
- 2000 prompt tok, 2000 gen tok
- 50 prompt tok, 1 gen tok



NVIDIA A100 40GB

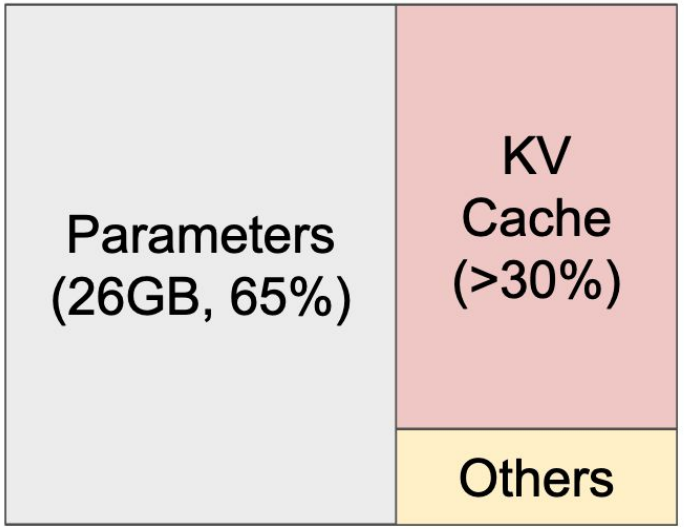
Image: arxiv.org/pdf/2309.06180

KV-Caches

Question: how many parallel requests we can serve if we have 20Gb of extra DRAM and one token takes up 200kb of cache?
Answer: it depends…

What if requests were very diverse:

- 2000 prompt tok, 1 gen tok
- 2000 prompt tok, 2000 gen tok
- 50 prompt tok, 1 gen tok



NVIDIA A100 40GB

Image: arxiv.org/pdf/2309.06180

Paged Attention

Question: how many parallel requests we can serve if we have 20Gb of extra DRAM and one token takes up 200kb of cache?
Answer: ~100000 tokens with Paged Attention

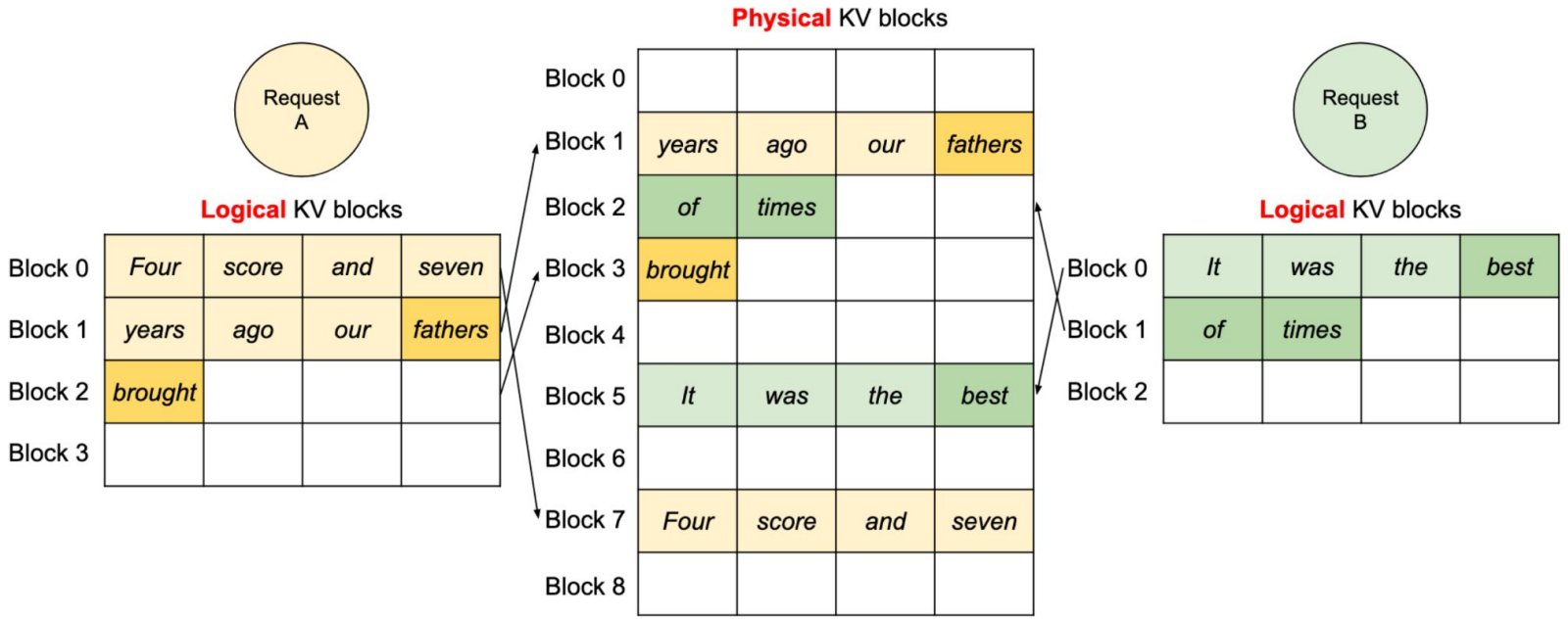


Image: arxiv.org/pdf/2309.06180

Inference Server

TLDR:

- Prioritizing throughput
- Many concurrent users with diverse requests
- Smart KV-Cache management is a must

Bottlenecks in GPU Inference

The math behind GPU utilization.

K tokens at a time:

- **Memory transfer** load of **O(N^2 + KxN)** per matrix-vector product.
- **Compute** load of **O(KxN^2)** per matrix-vector product.

Time cost of GPU operations:

- Bring million numbers from memory into kernels - **SLOW**
- Multiply million numbers with kernels - **FAST**

Bottlenecks in GPU Inference

K≈1 token at a time:

- The primary load in single user **chat applications**.
- **Memory transfer** load of **O(N^2)** per matrix-vector product.
- **Compute** load of **O(KxN^2)** per matrix-vector product.
- Memory transfer bottlenecked.

K>N tokens at a time:

- Prompt preprocessing, **highly parallel** inference.
- **Memory transfer** load of **O(KxN)** per matrix-matrix product.
- **Compute** load of **O(KxN^2)** per matrix-matrix product.
- Compute bottlenecked.

Parallel Inference - More Throughput

K≈1 token at a time:

- Latency doesn’t depend on K!

K>N tokens at a time:

- Latency is linear with K.

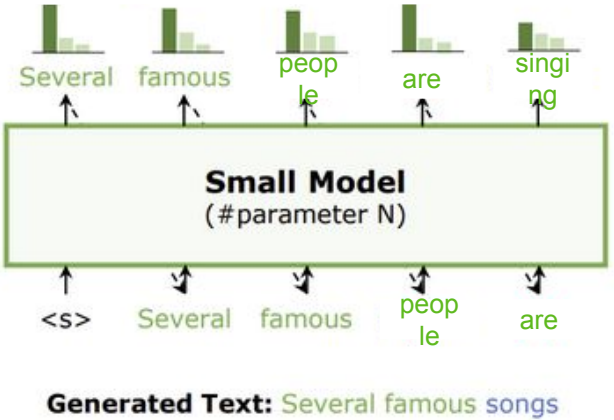
Speculative Decoding

https://arxiv.org/abs/2211.17192

Two models: main model (~Llama-70B) and draft model (~Llama-7B)

Greedy decoding:

Step 1: generate with draft model (sequential)



Speculative Decoding

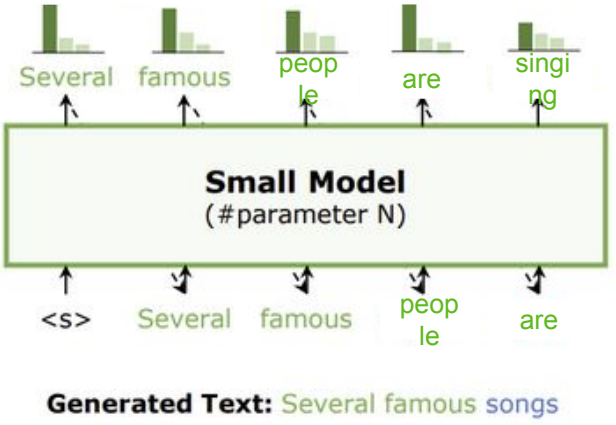
https://arxiv.org/abs/2211.17192

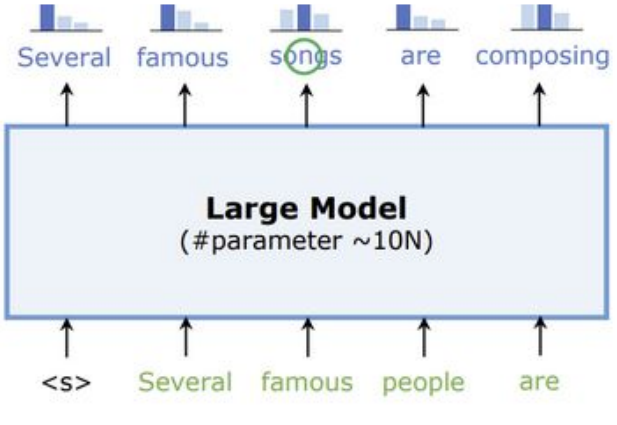
Two models: main model (~Llama-70B) and draft model (~Llama-7B)

Greedy decoding:

Step 1: generate with draft model (sequential)

Step 2: verify with large model (parallel)





Speculative Decoding

https://arxiv.org/abs/2211.17192

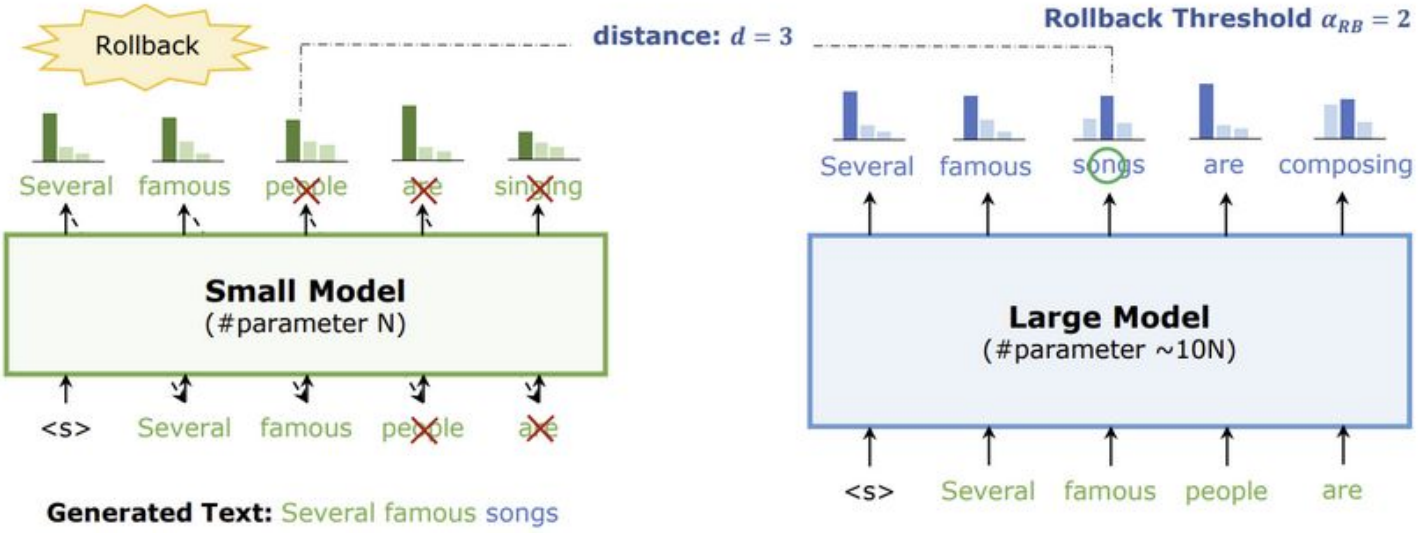
Two models: main model (~Llama-70B) and draft model (~Llama-7B)

Greedy decoding:

Step 1: generate with draft model (sequential)

Step 2: verify with large model (parallel)

Accept multiple tokens



Speculative Decoding

https://arxiv.org/abs/2211.17192

Two models: main model (~Llama-70B) and draft model (~Llama-7B)

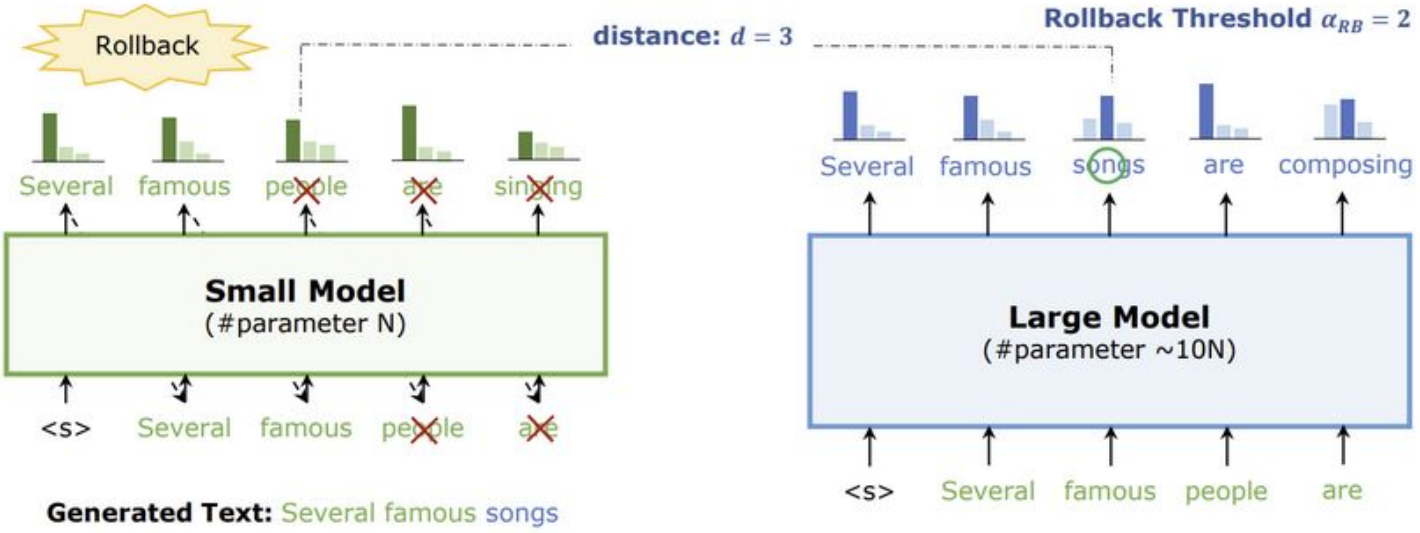
Greedy decoding:

Step 1: generate with draft model (sequential)

Step 2: verify with large model (parallel)

Accept multiple tokens

Repeat

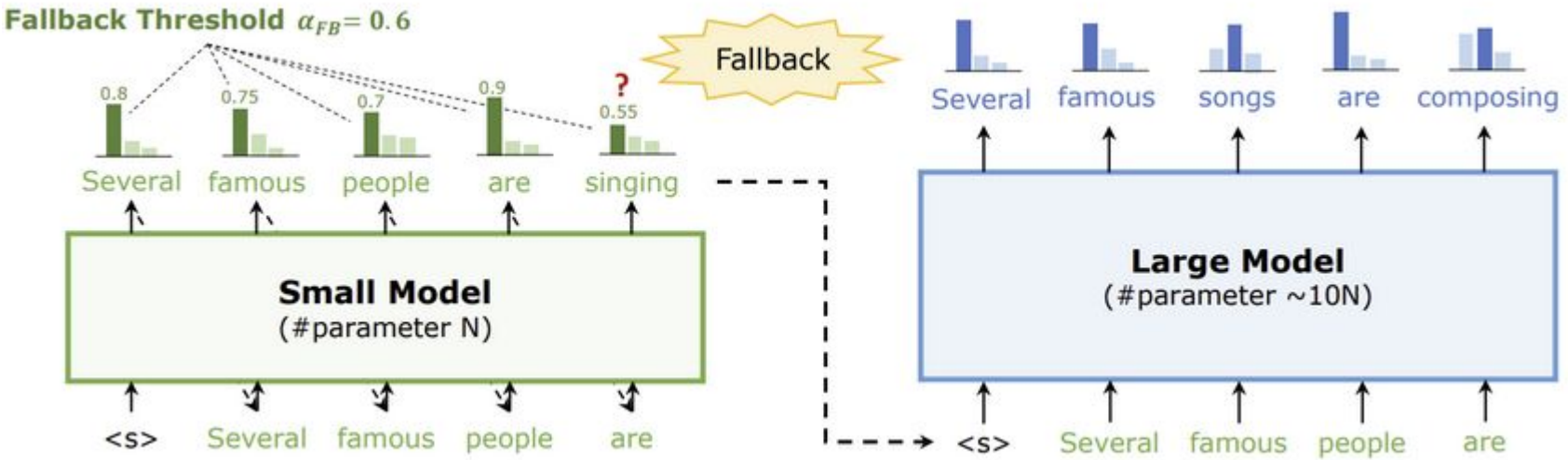


Speculative Decoding

https://arxiv.org/abs/2211.17192

Two models: main model (~Llama-70B) and draft model (~Llama-7B)

Sampling (temperature, top-p, top-k): generate, then reject with probability sampling probability proven equal to regular sampling

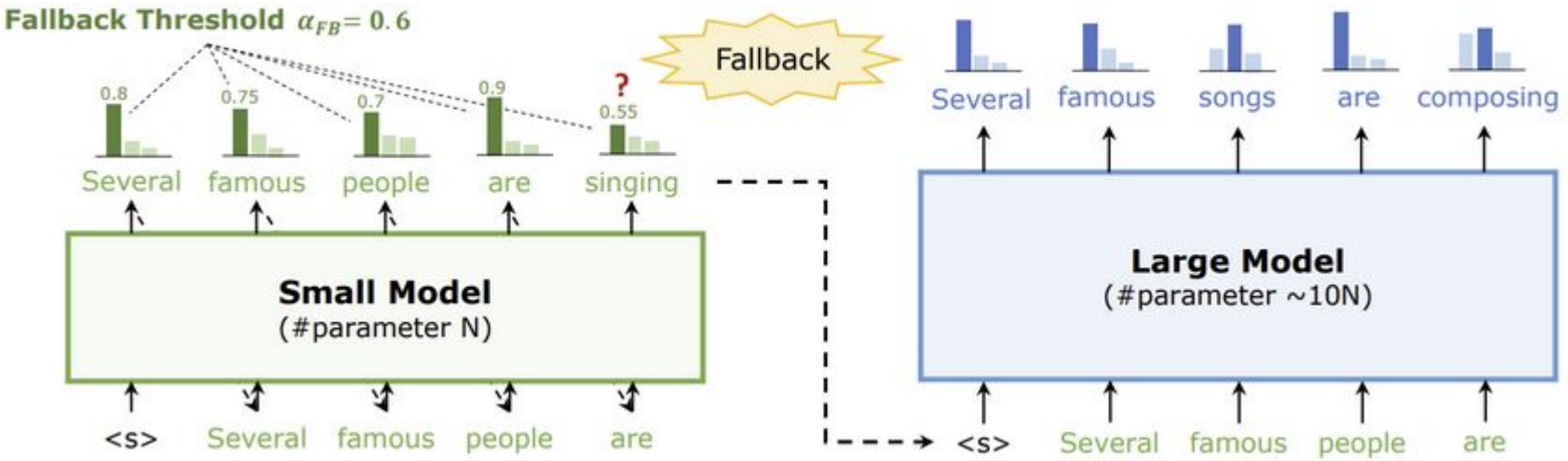


Speculative Decoding

That you'll implement

Two models: main model (Llama-8B) and draft model (of your choice)

Greedy Sampling: generate, then reject everything after exact match



Mobile Deployment

- Run model on a mobile device
- Features:
  - Diverse devices (or not if you're Apple)
  - New priority: *power consumption*
  - Specialized matmul kernels

Priorities:

Mobile Deployment

TensorFlow.js	All modern browsers
CoreML	iOS devices
NNAPI	Android devices
ExecuTorch	iOS/Android/Embedded
MLX	Apple Silicon

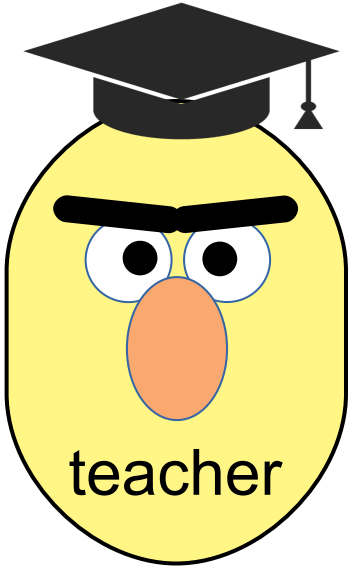
Summary

TLDR:

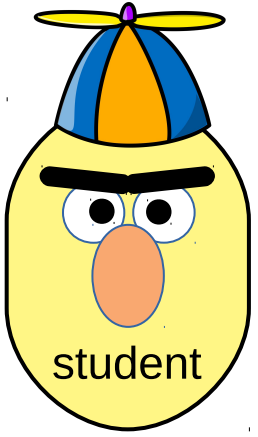
- For any system with LLMs, there's a bottleneck, e.g.:
  - Max throughput for inference servers
  - Memory/latency/power for mobile deployment

Chapter 2: How Do I Improve My Model?

Compression by Distillation

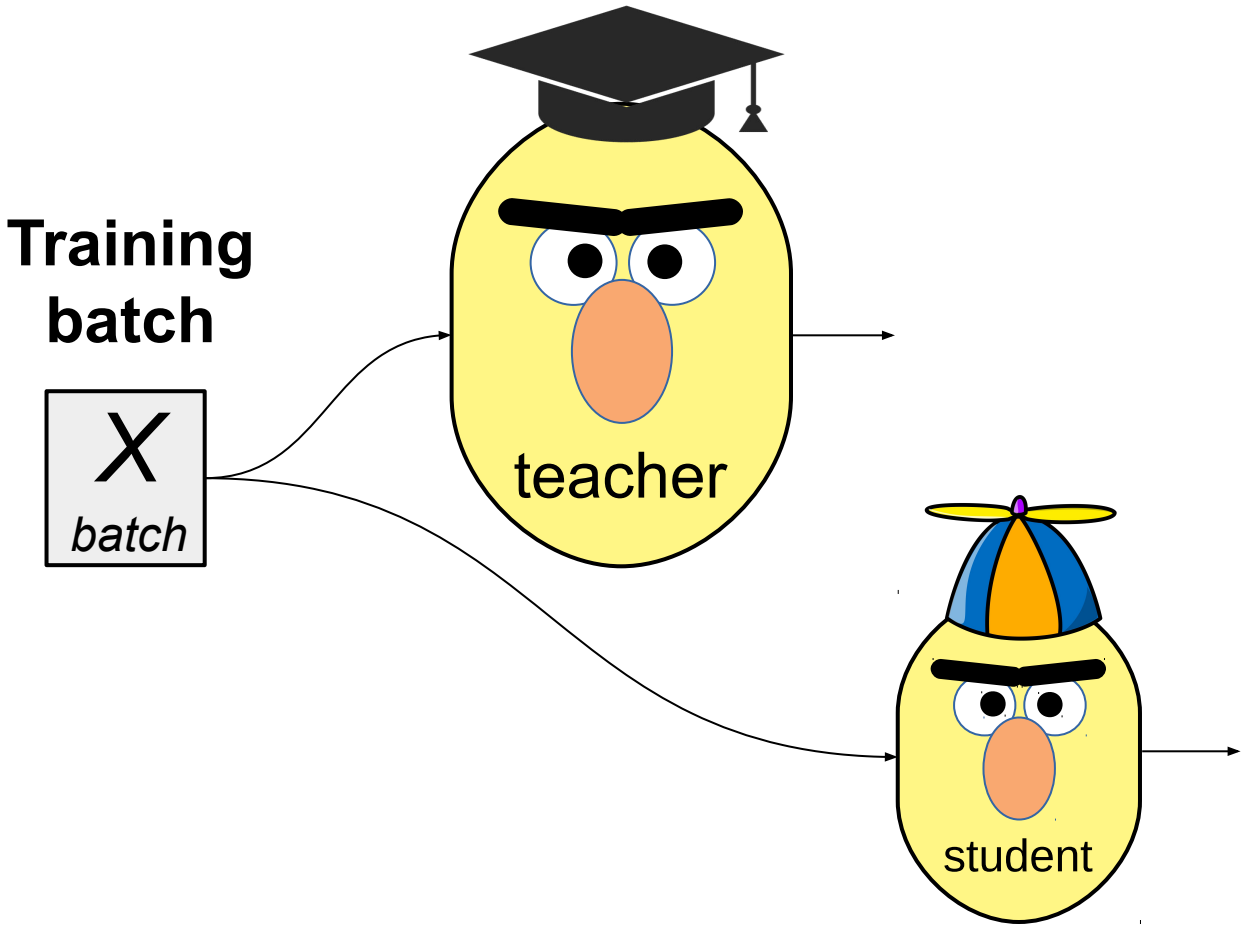


First, get the best performing model regardless of size

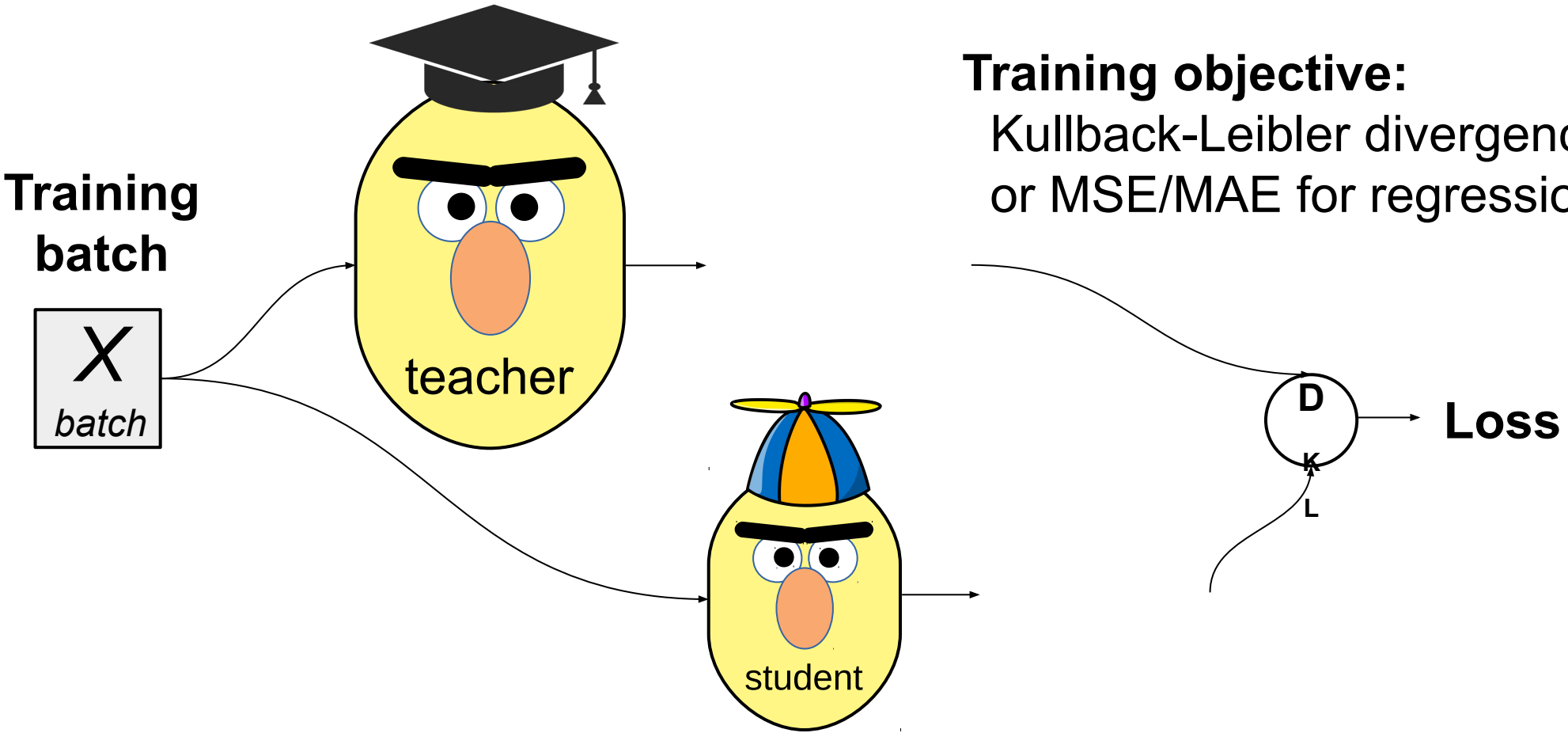


Then, train a more compact model to approximate it!

Compression by Distillation



Compression by Distillation



Compression by Distillation

- Student architecture choices:
  Naïve: same but smaller, less layers / hidden units
  - e.g. DistillBERT: https://arxiv.org/pdf/1910.01108.pdf
  - Same as BERT-base, but with
  - *half as many layers*
  - (and ≈1.5 times faster)

Model	# param. (Millions)	Inf. time (seconds)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

Model	**Score**	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

Compression by Distillation

- Student architecture choices:
  Naïve: same but smaller, less layers / hidden units, random init
- **Sparse along layers/attn_heads/mlp_neurons:** sparsify but obtain dense representation, init from teacher

  Let’s focus on that…

Minitron Approach

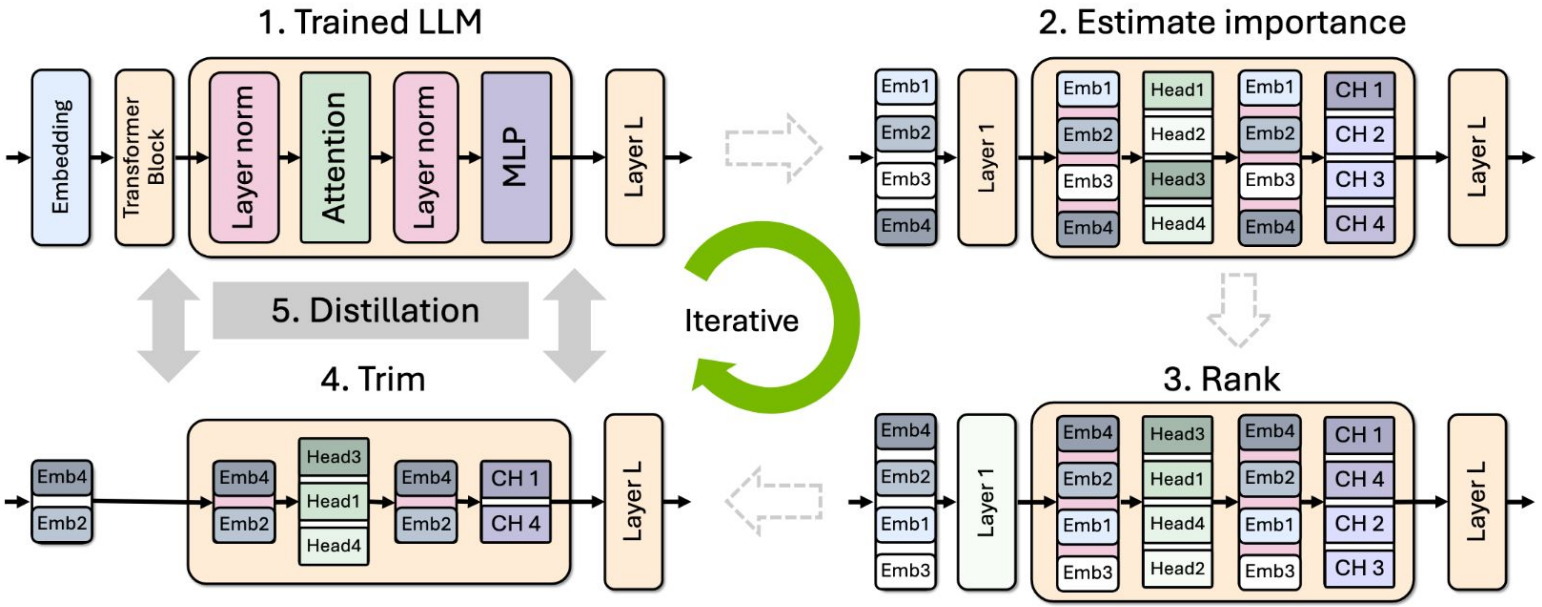
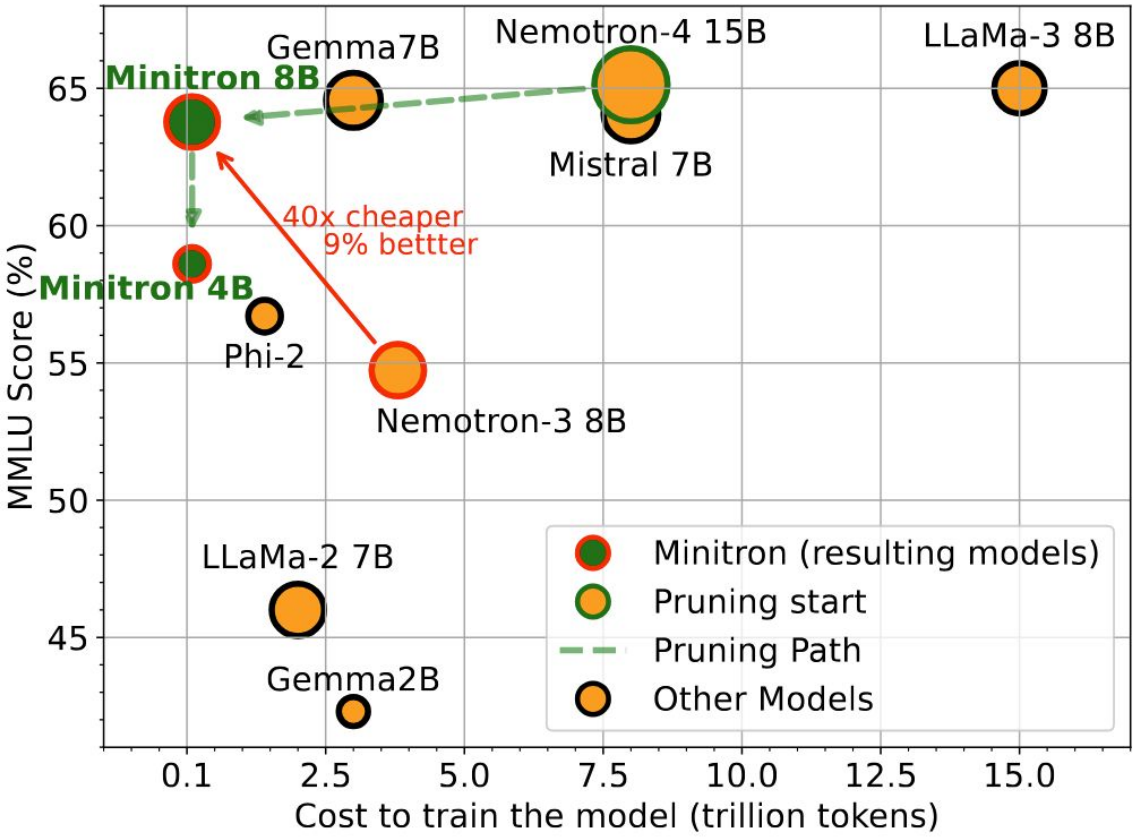


Image: arxiv.org/pdf/2407.14679

Figure 2: High-level overview of our proposed iterative pruning and distillation approach to train a family of smaller LLMs. On a pretrained LLM, we first evaluate importance of neurons, rank them, trim the least important neurons and distill the knowledge from the original LLM to the pruned model. The original model is replaced with the distilled model for the next iteration of compression.

Minitron Approach



Source: arxiv.org/pdf/2407.14679

Minitron Approach

1. To train a family of LLMs, train the largest one and prune+distill iteratively to smaller LLMs.
2. Use (batch=L2, seq=mean) importance estimation for width axes and PPL/BI for depth.
3. Use single-shot importance estimation; iterative provides no benefit.
4. Prefer width pruning over depth for the model scales we consider (≤ 15B).
5. Retrain exclusively with distillation loss using KLD instead of conventional training.
6. Use (logit+intermediate state+embedding) distillation when depth is reduced significantly.
7. Use logit-only distillation when depth isn't reduced significantly.
8. Prune a model closest to the target size.
9. Perform lightweight retraining to stabilize the rankings of searched pruned candidates.
10. If the largest model is trained using a multi-phase training strategy, it is best to prune and retrain the model obtained from the final stage of training.

Source: arxiv.org/pdf/2407.14679

When Distillation is Worth it

- LLMs are trained up to compute budget
- Training the teacher (larger) model is VERY expensive
- Makes sense where:

When Distillation is Worth it

- LLMs are trained up to compute budget
- Training the teacher (larger) model is VERY expensive
- Makes sense where:
  - Data is scarce, compute is not

When Distillation is Worth it

- LLMs are trained up to compute budget
- Training the teacher (larger) model is VERY expensive
- Makes sense where:
  - Data is scarce, compute is not
  - A single teacher trains multiple students

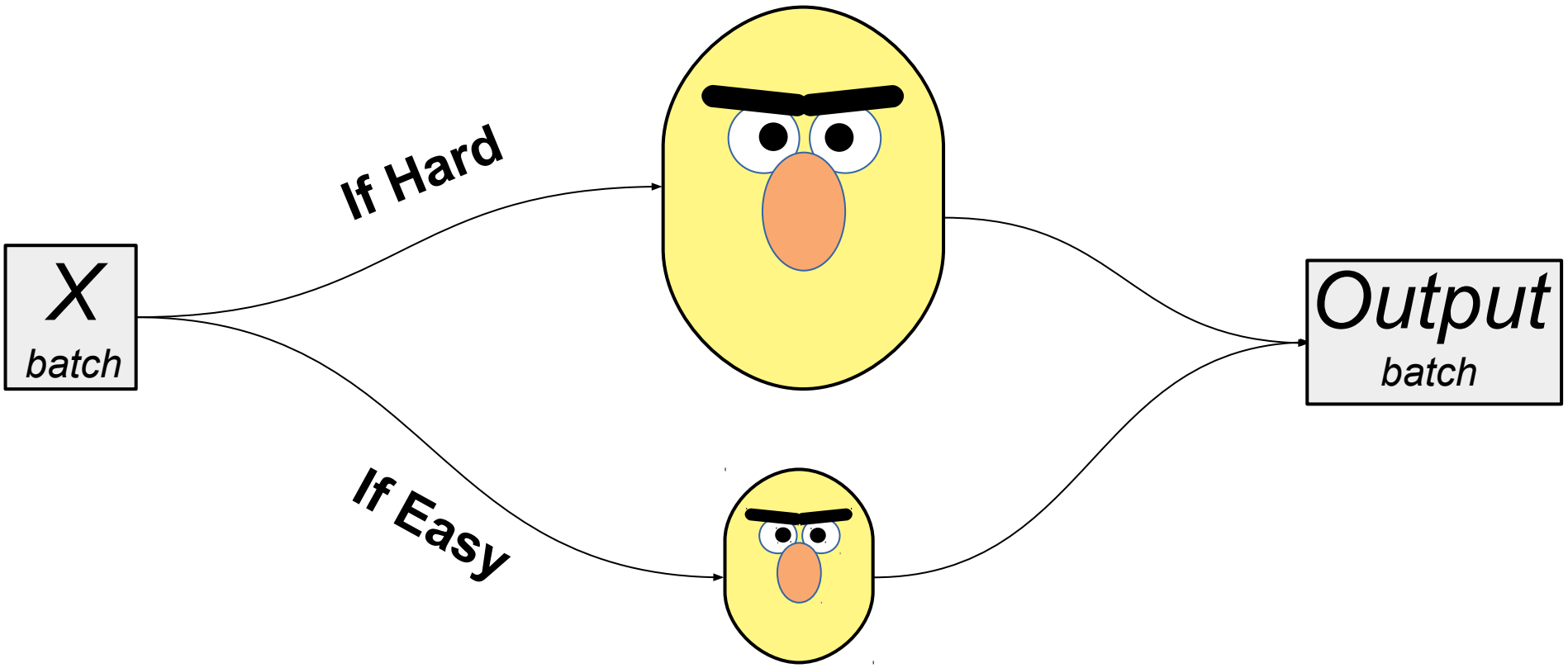
When Distillation is Worth it

- LLMs are trained up to compute budget
- Training the teacher (larger) model is VERY expensive
- Makes sense where:
  - Data is scarce, compute is not
  - A single teacher trains multiple students
  - Teacher achieves generalization and smaller model doesn't

Read more:

- Distillation Scaling Laws
- Puzzle: Distillation-Based NAS for Inference-Optimized LLMs

What to do with the teacher?



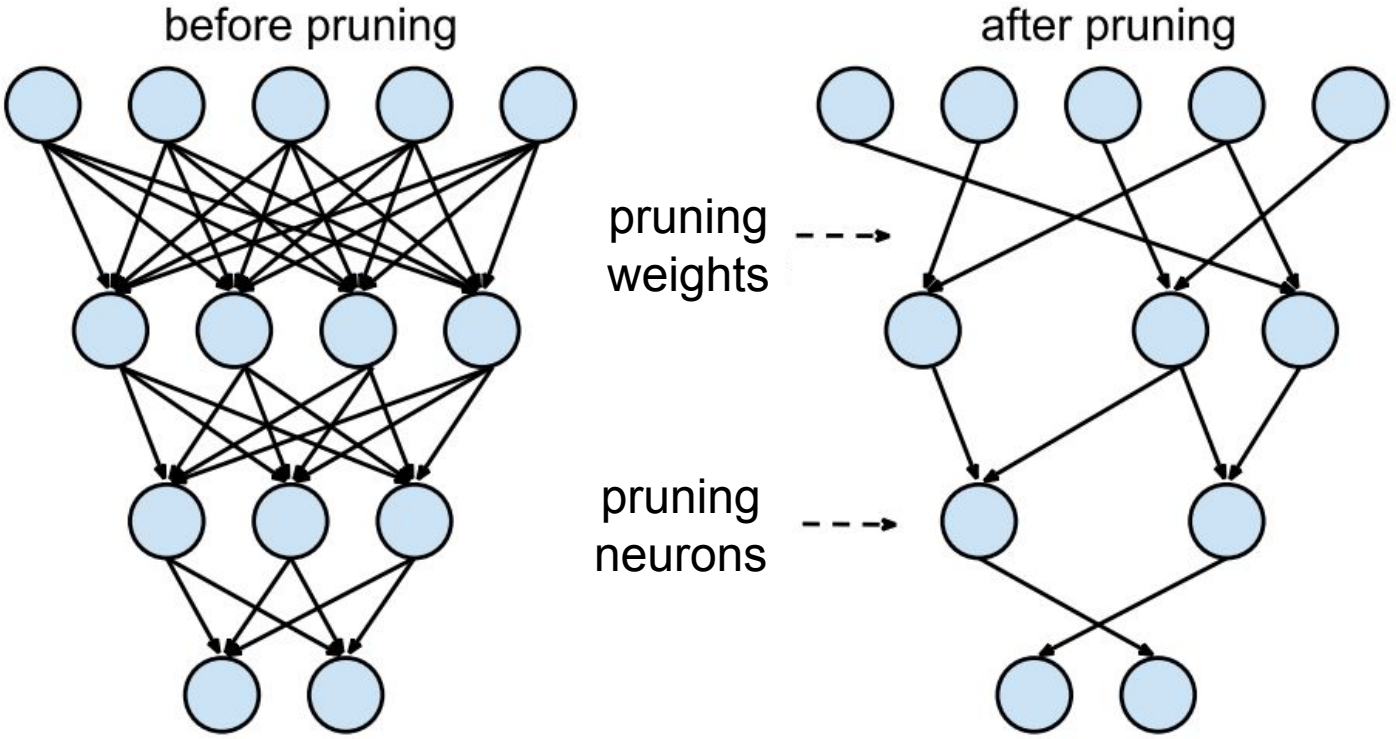
Going Inside Model Layers

Compression by Sparsification

Do we really need all D by D weights?

Compression by Pruning

Do we really need all D by D weights?



Magnitude Pruning

Drop ~5% smallest weights
from each layer every 1000 steps
(and keep training)

Importance Estimation

Minitron:

- attn_head/neuron level: activation scales
- Block level: PPL

L0: Learnable importance estimation:

$$w_i = w_i \times \sigma(a_i + N(0,1))$$

Read more: https://arxiv.org/abs/1712.01312
Alternative: https://arxiv.org/abs/1701.05369

Compression by Sparsification

Unstructured sparsity = prune individual weights (minimal model size)

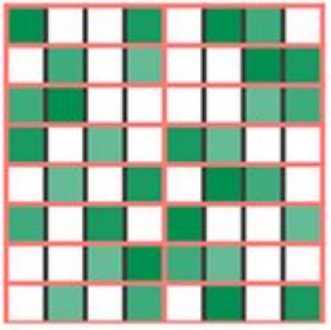
Structured sparsity= prune entire neurons/heads (fastest inference)

Compression by Sparsification

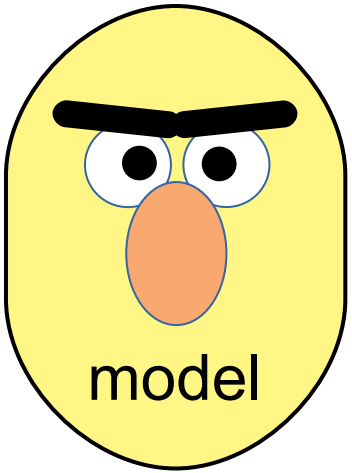
Unstructured sparsity = prune individual weights (minimal model size)

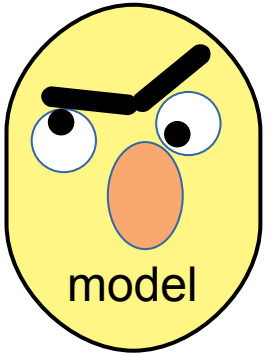
Structured sparsity= prune entire neurons/heads (fastest inference)

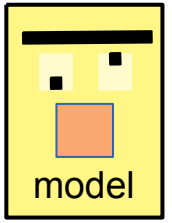
Note: some GPU/FPGAs also run fast with low-level structured sparsity, e.g. “Any 2 of 4 consecutive weights” (left)



Compression by quantization

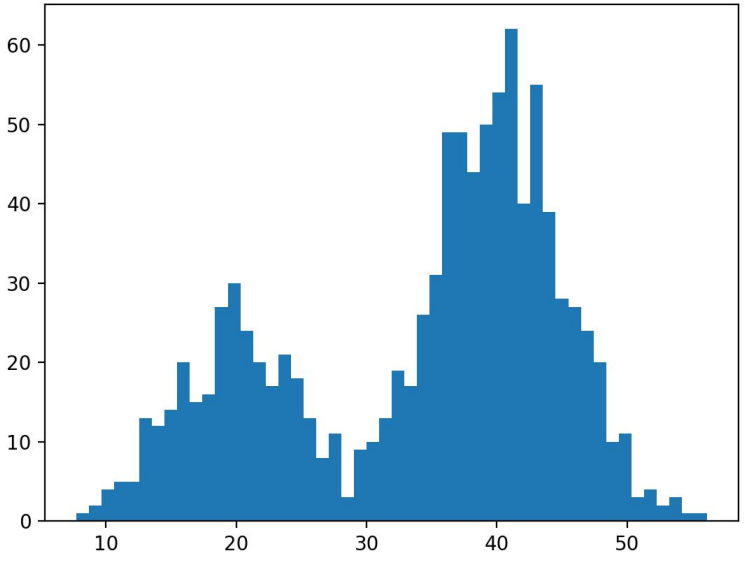






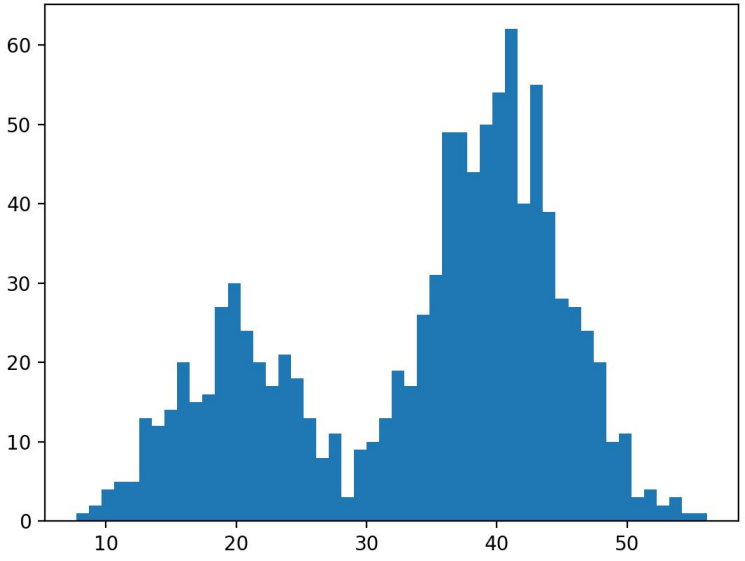
Quantization basics

Goal: encode data as int8 / int4



Quantization basics

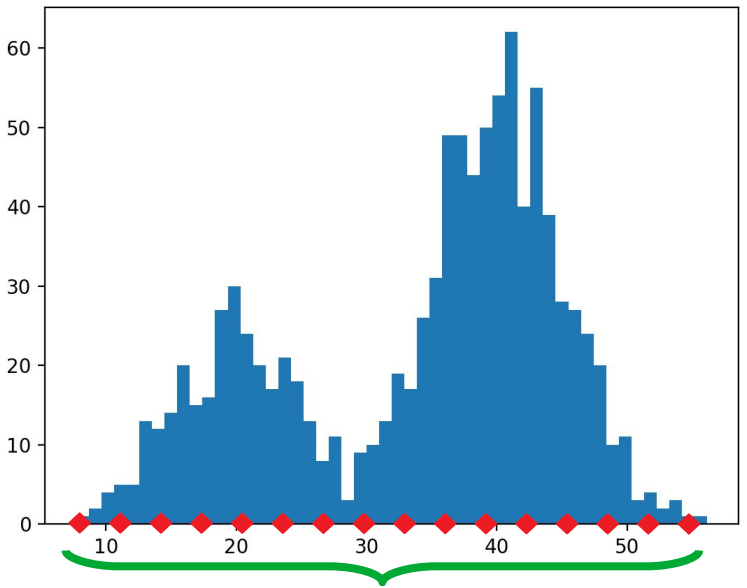
Goal: encode data as int8 / int4



not an ideal range for int4

Linear quantization

Fit a linear range to data



scale = (max(w) – min(w)) / 2^4

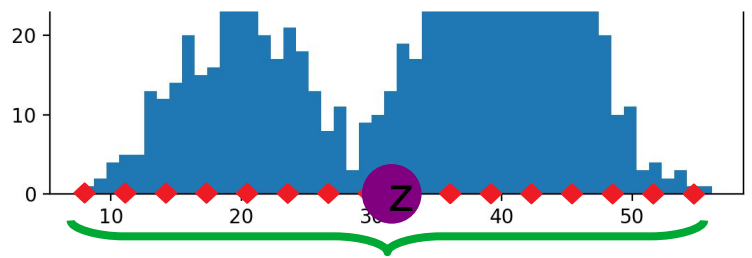
zero = -min(w) / scale

Linear quantization

Fit a linear range to data

Encode: $\mathbf{c_i} = (w_i / \mathbf{s} + \mathbf{z}).\text{clip}(0, 15)$ uint4 range

Decode: $\mathbf{w_i}$ = ??? ideas?



scale = (max(w) – min(w)) / 2^4

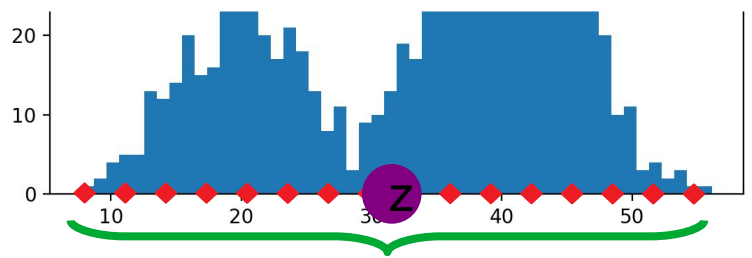
zero = -min(w) / scale

Linear quantization

Fit a linear range to data

Encode: $\mathbf{c_i} = (w_i / \mathbf{s} + \mathbf{z}).\text{clip}(0, 15)$ uint4 range

Decode: $\mathbf{w_i} \approx \mathbf{s} * \mathbf{c_i} - \mathbf{z}$



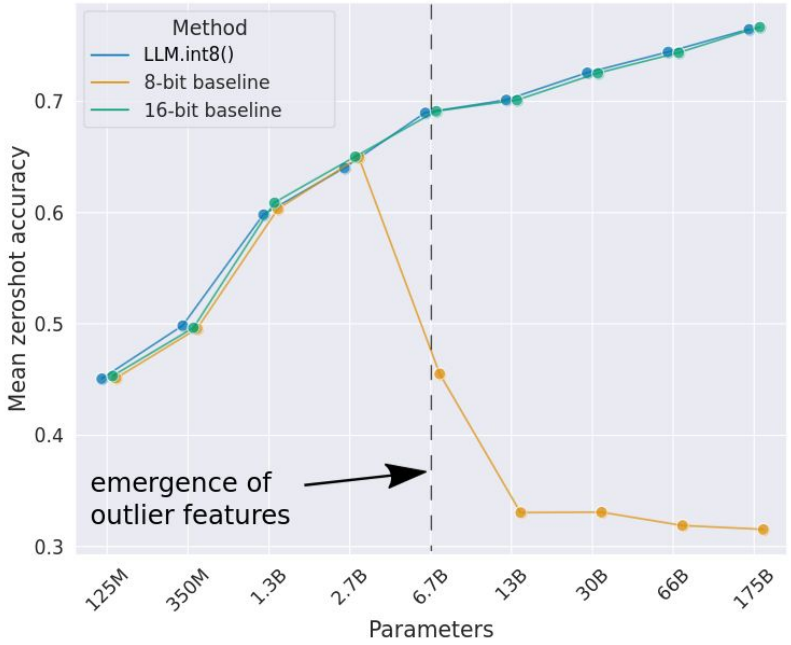
scale = (max(w) – min(w)) / 2^4

zero = -min(w) / scale

LLM.8bit(): some weights are more important

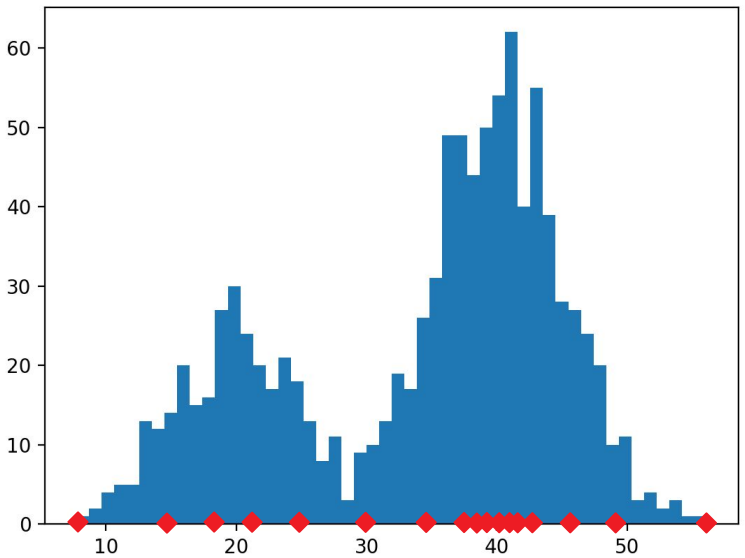
https://arxiv.org/abs/2208.07339

TL;DR in very LLM, some input features become outliers
Weights for those features are sensitive
KEEP <1% MOST SENSITIVE WEIGHTS IN 16-bit!



Nonlinear quantization

Fit codes to data



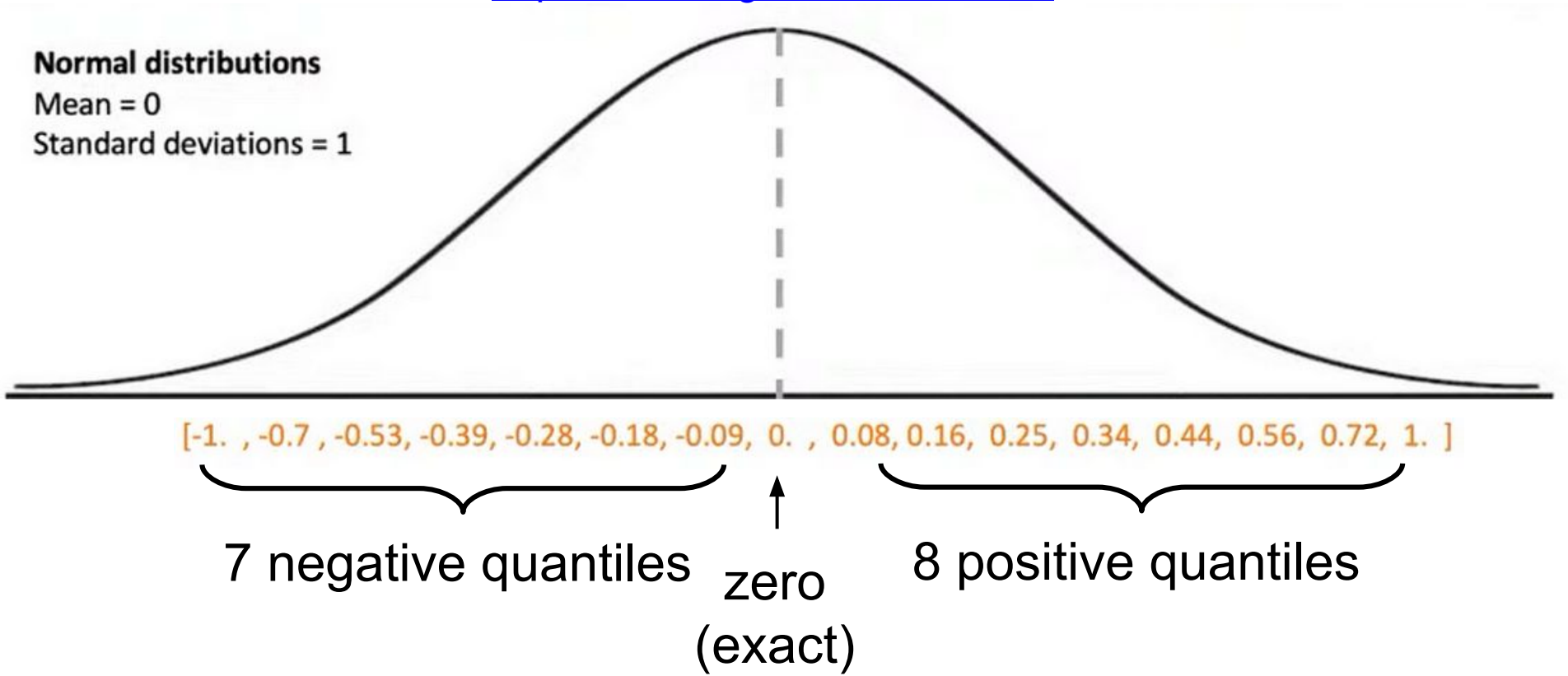
Compute a grid of percentiles or centroids (k-means 1d)

Store each weight as the index of nearest percentile/centroid

Static nonlinear case: NF4

https://arxiv.org/abs/2305.14314

https://arxiv.org/abs/2306.06965



Static nonlinear case: NF4

https://arxiv.org/abs/2305.14314

https://arxiv.org/abs/2306.06965

How to use:

```
1 model = transformers.AutoModelForCausalLM.from_pretrained(
2     "Enoch/llama-7b-hf", load_in_4bit=True)
```

[-1. , -0.7 , -0.53, -0.39, -0.28, -0.18, -0.09, 0. , 0.08, 0.16, 0.25, 0.34, 0.44, 0.56, 0.72, 1.]

7 negative quantiles

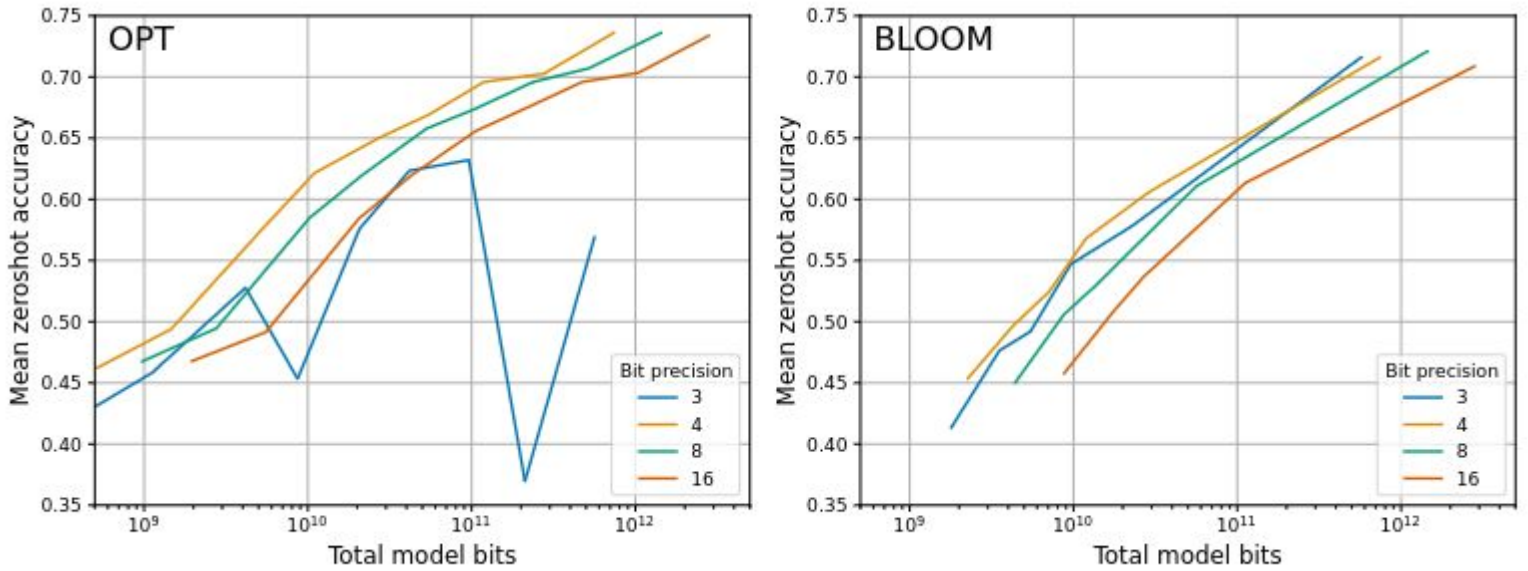
zero (exact)

8 positive quantiles

How many bits is best?

https://arxiv.org/abs/2212.09720

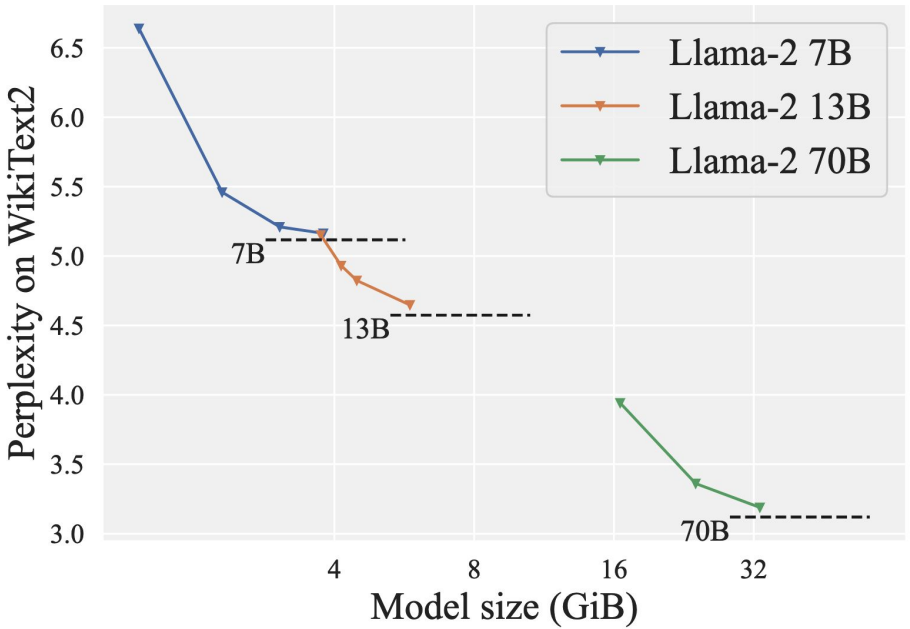
TL;DR 3-4 bits looks optimal
2-bit: Smaller models in 4 bits are better than larger models in 2 bits



How many bits is best?

https://arxiv.org/abs/2401.06118

TL;DR ~2 bits looks optimal
2-bit: more expensive to quantize, more expensive to run



GPU Inference

K≈1 token at a time:

- The primary load in single user **chat applications**.
- **Memory transfer** load of **O(N^2)** per matrix-vector product.
- **Compute** load of **O(KxN^2)** per matrix-vector product.
- Memory transfer bottlenecked.

K>N tokens at a time:

- Prompt preprocessing, **highly parallel** inference.
- **Memory transfer** load of **O(KxN)** per matrix-matrix product.
- **Compute** load of **O(KxN^2)** per matrix-matrix product.
- Compute bottlenecked.

Quantized GPU Inference

With weight compression ratio **C**.

K≈1 token at a time:

- The primary load in single user **chat applications**.
- **Memory transfer** load of **O(N^2 / C)** per matrix-vector product.
- **Compute** load of **O(KxN^2)** per matrix-vector product.
- Memory transfer bottlenecked.

K>N tokens at a time:

- Prompt preprocessing, **highly parallel** inference.
- **Memory transfer** load of **O(KxN)** per matrix-matrix product.
- **Compute** load of **O(KxN^2)** per matrix-matrix product.
- Compute bottlenecked.

Quantized GPU Inference

With weight compression ratio **C**.

K≈1 token at a time:

- **Memory transfer** load of **O(N^2 / C)** per matrix-vector product.
- Memory transfer bottlenecked.

K>N tokens at a time:

- **Compute** load of **O(KxN^2)** per matrix-matrix product.
- Compute bottlenecked.

Quantized GPU Inference

With weight compression ratio **C**.

K≈1 token at a time:

- **C** times faster inference!

K>N tokens at a time:

- No speedup :(

Fully Quantized GPU Inference

With weight+activations compression ratio **C**.

K≈1 token at a time:

- **Memory transfer** load of **O(N^2 / C)** per matrix-vector product.
- Memory transfer bottlenecked.

K>N tokens at a time:

- **Compute** load of **O(KxN^2 / C)** per matrix-matrix product.
- Compute bottlenecked.

Fully Quantized GPU Inference

With weight+activations compression ratio **C**.

K≈1 token at a time:

- **C** times faster inference!

K>N tokens at a time:

- **C** times faster inference!

Fully Quantized GPU Inference

Weight-only Quantization:

- Faster only for small batch sizes.

Weight+Activation Quantization:

- Faster for any batch sizes!

Fully Quantized GPU Inference

Weight-only Quantization:

- Faster only for small batch sizes.
- Needs **KERNELS**:
  - Marlin W4A16 in vLLM
  - Bunch of stuff at llama.cpp

Weight+Activation Quantization:

- Faster for any batch sizes!
- Needs **HARDWARE** support:
  - INT4 on Ada GPUs
  - FP8 on Hopper GPUs
  - NVFP4/MXFP4/MXFP8 on Blackwell GPUs (see this paper)

Post-Training Quantization

Can we take advantage of **a few** data points
without training the model?

Post-Training Quantization

Can we take advantage of **a few** data points
without training the model?

- find which weights are multiplied by larger inputs
- find correlated or anti-correlated weights
- try to “cancel out” quantization errors

GPTQ

Frantar et al, 2022

Layer-wise quadratic objective $\operatorname{argmin}_{\widehat{\mathbf{W}}_\ell} \quad ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

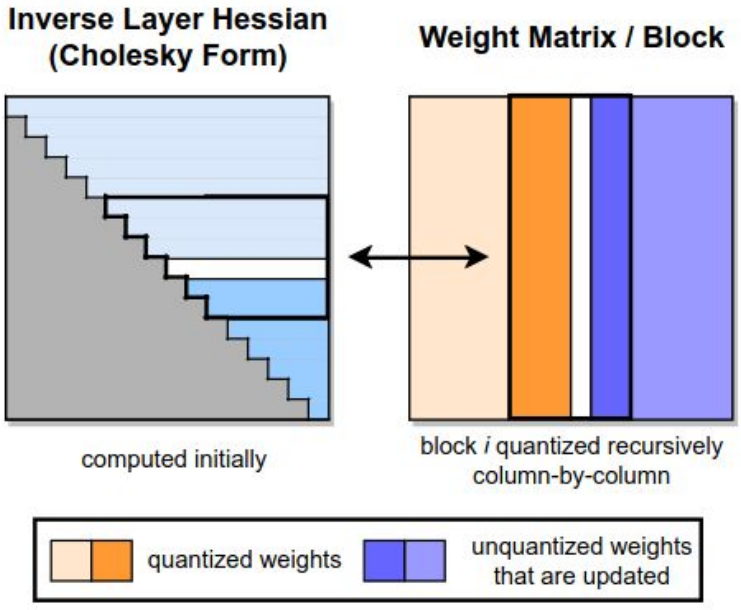
GPTQ

Frantar et al, 2022

Layer-wise quadratic objective $\operatorname{argmin}_{\widehat{\mathbf{W}}_\ell} \; ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

For i = 1 … in_features:

- **quantize** i-th column of weight matrix
  (from one input feature and all outputs)
- freeze the quantized model forever
- update all remaining columns



GPTQ

Frantar et al, 2022

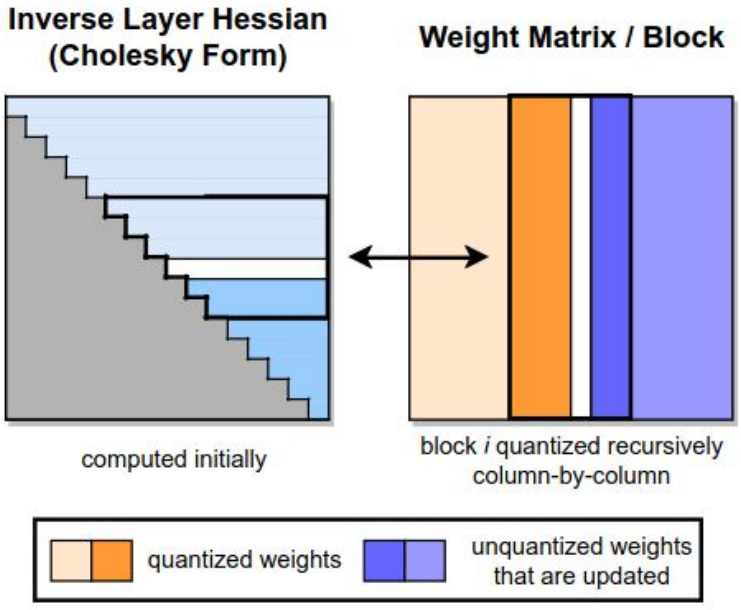
Minimize the same objective $\text{argmin}_{\widehat{\mathbf{W}}_\ell} \quad ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

For i = 1 … in_features:

- **quantize** i-th column of weight matrix
  (from one input feature and all outputs)
- freeze the quantized model forever
- update all remaining columns

Use linear quantization with one scale & zero per each group of G weights

Tricks: process weights in “hacky” order



GPTQ

[Frantar et al, 2022](#)

Minimize the same objective $\text{argmin}_{\widehat{\mathbf{W}}_\ell} \; ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

For i = 1

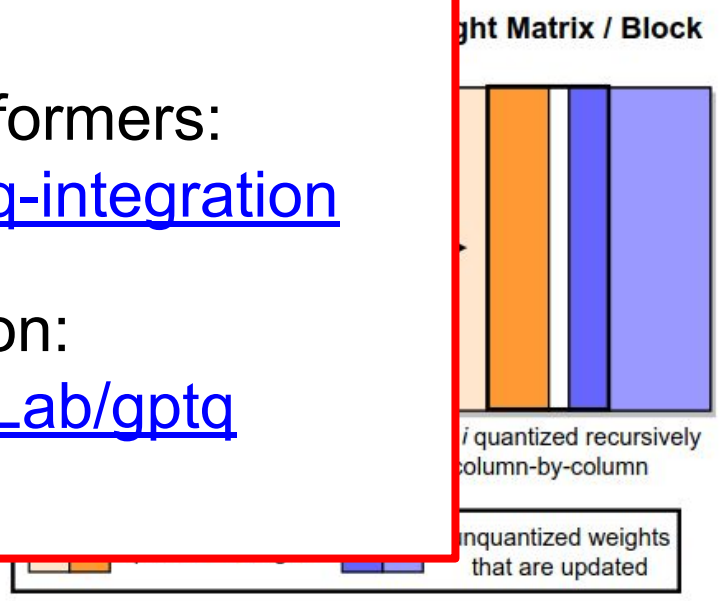
- **quantiz**
  (from
- freeze t
- update

Use linear

zero per ea

Native support in HF Transformers:
https://huggingface.co/blog/gptq-integration

Original implementation:
https://github.com/IST-DASLab/gptq



Tricks: process weights in “hacky” order

SparseGPT

Frantar et al, 2023

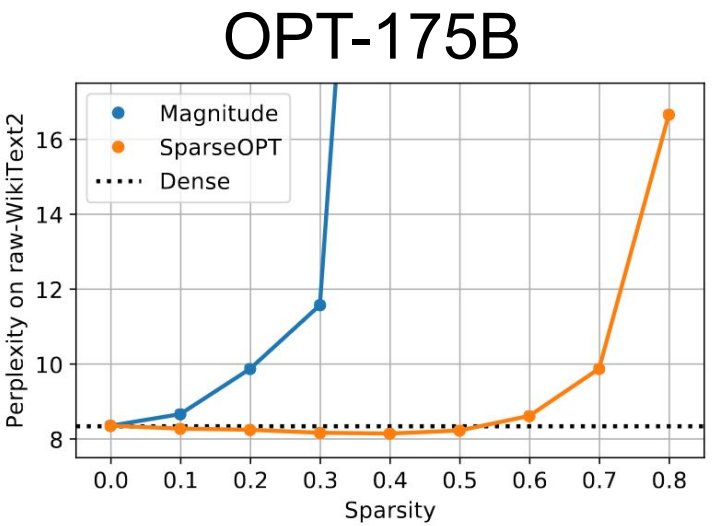
Minimize the same objective $\text{argmin}_{\widehat{\mathbf{W}}_\ell} \quad ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

For i = 1 … in_features:

- **sparsify** i-th column of weight matrix
  (from one input feature and all outputs)
- freeze the quantized model forever
- update all remaining columns

Dynamically choose how many weights to prune on each step (threshold on error)

Trick: can do 2-out-of-4 sparsity for A100



QuIP#/AQLM/SpinQuant

Minimize the same objective $\operatorname{argmin}_{\widehat{\mathbf{W}}_\ell} \quad ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

QuIP#/AQLM/SpinQuant

Minimize the same objective $\text{argmin}_{\widehat{\mathbf{W}}_\ell} \quad ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

But also finetune while quantizing!

QuIP#

Minimize the same objective $\operatorname{argmin}_{\widehat{\mathbf{W}}_\ell} \quad ||\mathbf{W}_\ell \mathbf{X}_\ell - \widehat{\mathbf{W}}_\ell \mathbf{X}_\ell||_2^2$

But also finetune while quantizing!

On the level of blocks, after quantizing each linear layer:

$$\operatorname{argmin}_{\text{Unquantized} \in B} \left\| B(X) - \widehat{B}(X) \right\|_2^2$$

More Quantization Papers

PV-Tuning – SOTA weight-only quantization optimization procedure
https://arxiv.org/abs/2405.14852

QTIP – SOTA weight-only quantization algorithm
https://arxiv.org/abs/2406.11235

QuaRot - SOTA GPTQ-based weight+act quantization algorithm
https://arxiv.org/abs/2404.00456

SpinQuant – SOTA weight+act quantization algorithm with finetuning
https://arxiv.org/abs/2405.16406

Many more cool papers

For Prod

LLM Compressor

https://github.com/vllm-project/llm-compressor

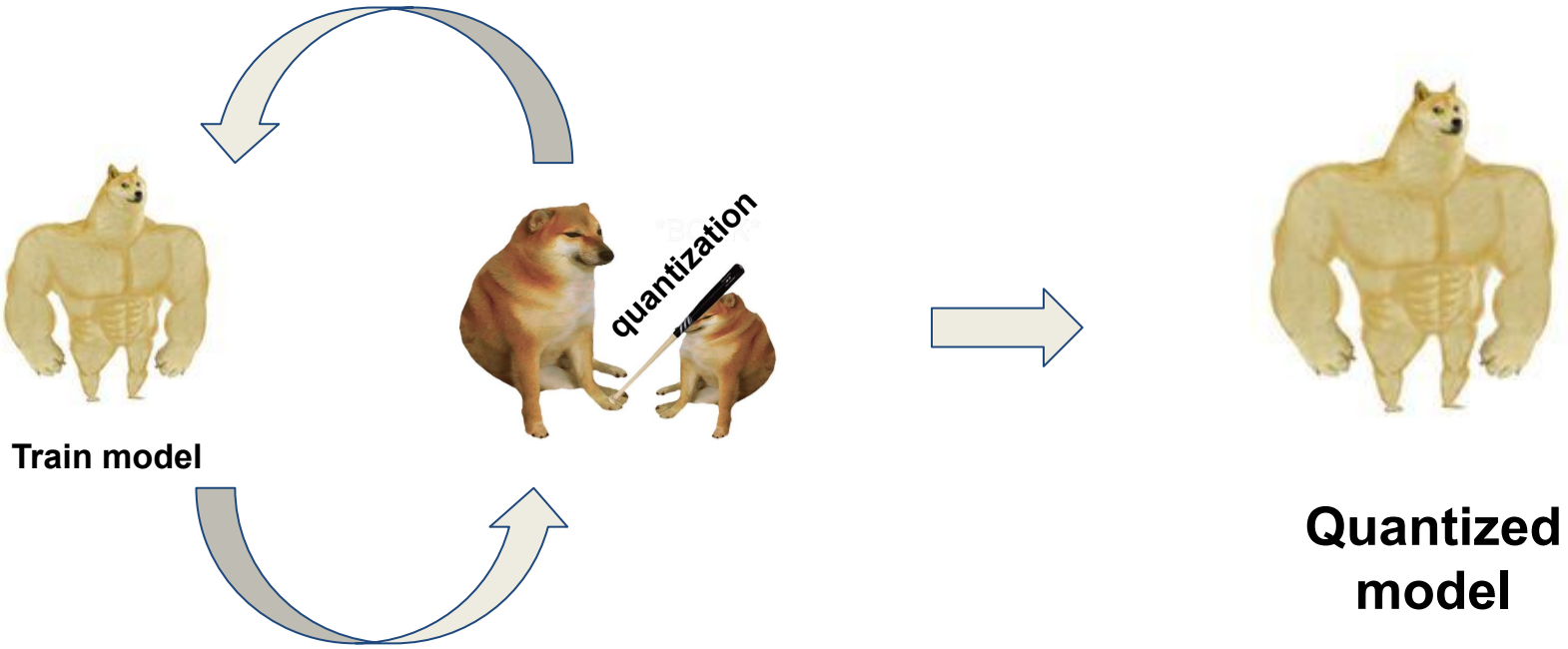
- Works with vLLM
- Real fast kernels
- Supports modern quantization techniques
- Supports all modern data-types and hardware

Post-Training Quantization

Train model

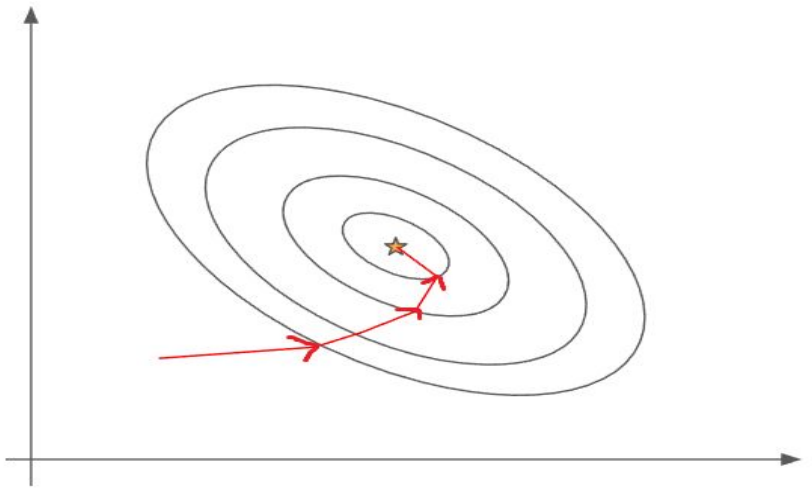
Quantized model

Quantization-Aware Training



Always Gradient-Based Optimization

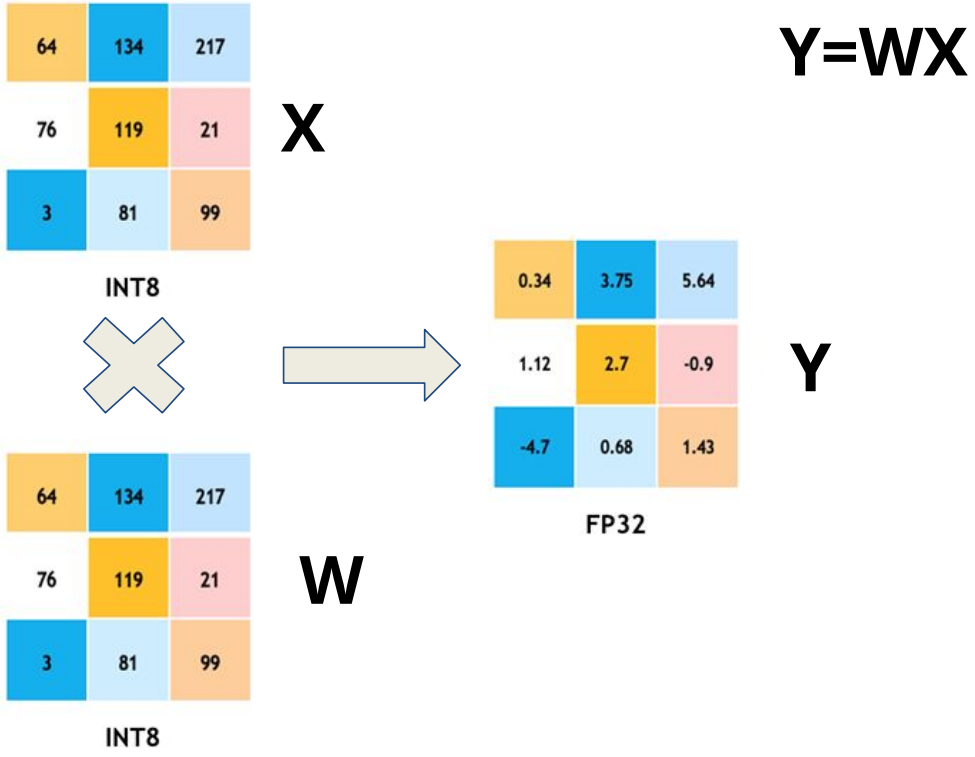
$$\begin{aligned}\hat{\boldsymbol{\phi}} &= \underset{\boldsymbol{\phi}}{\operatorname{argmin}}\left[\mathrm{L}[\boldsymbol{\phi}]\right] \\ &= \underset{\boldsymbol{\phi}}{\operatorname{argmin}}\left[\sum_{i=1}^{I}\ell_i[\mathbf{x}_i,\mathbf{y}_i]\right],\end{aligned}$$



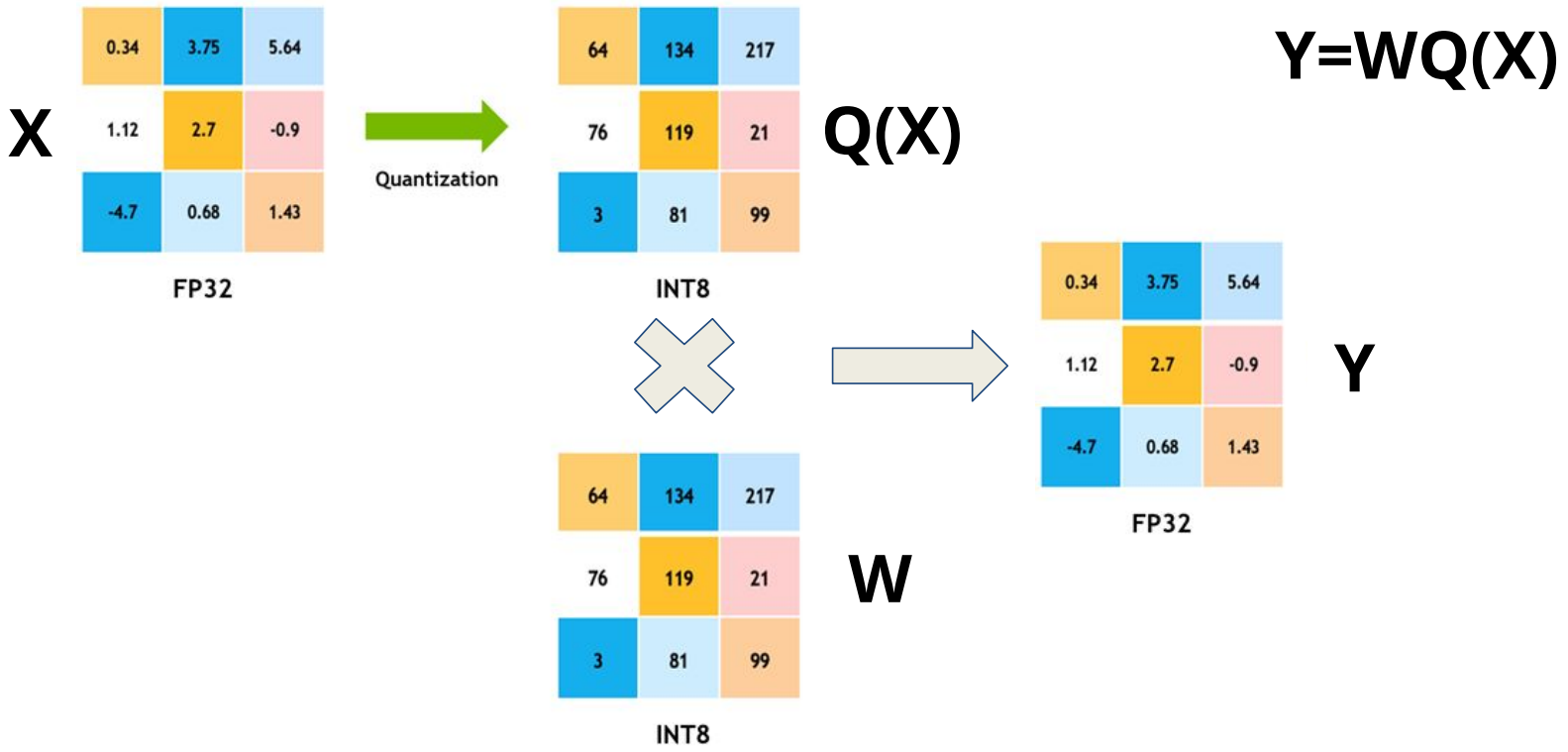
Gradient Descent

Image: computer.org/csdl/magazine/cs/2014/04/mcs2014040080/13rRUwhpBTR

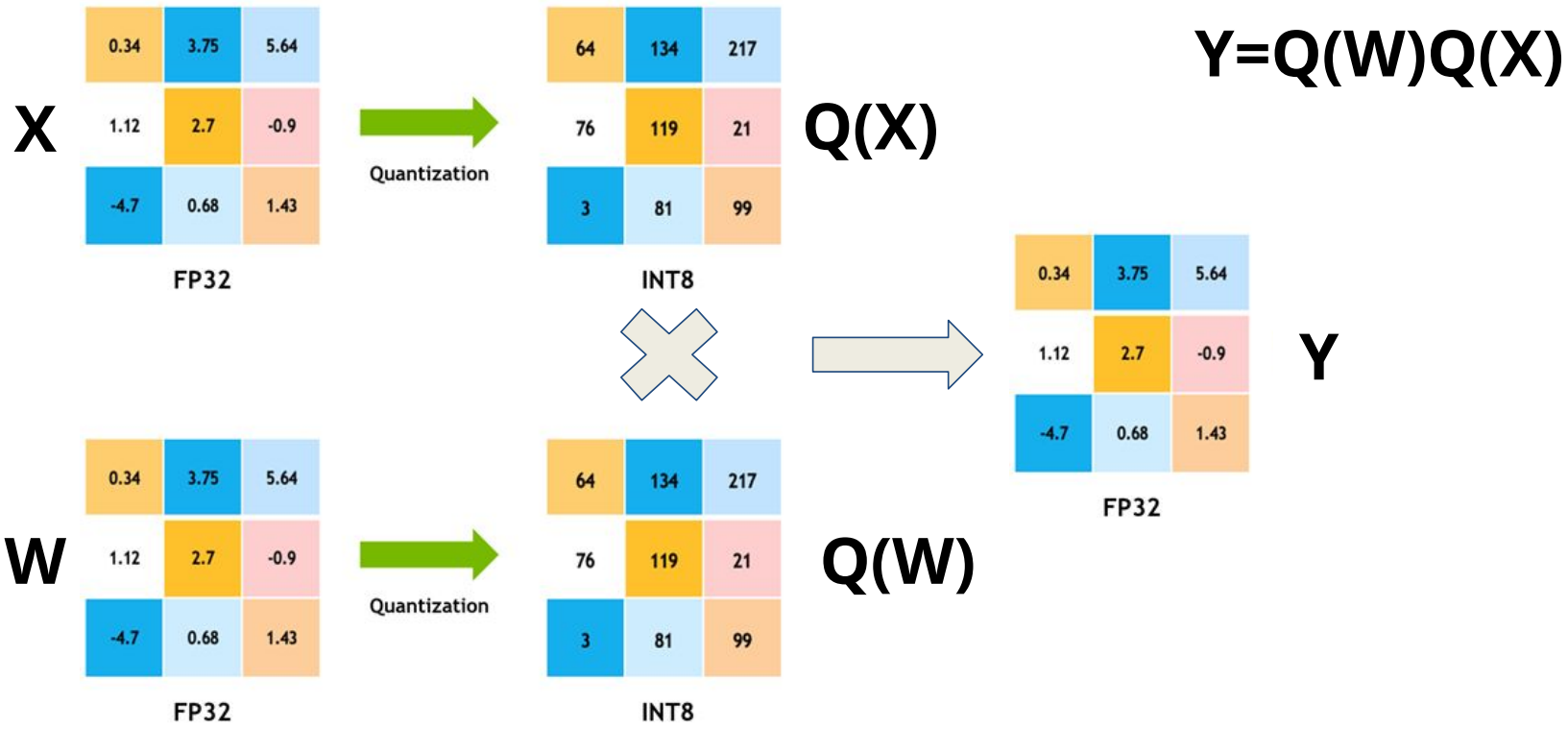
Quantized Forward Pass



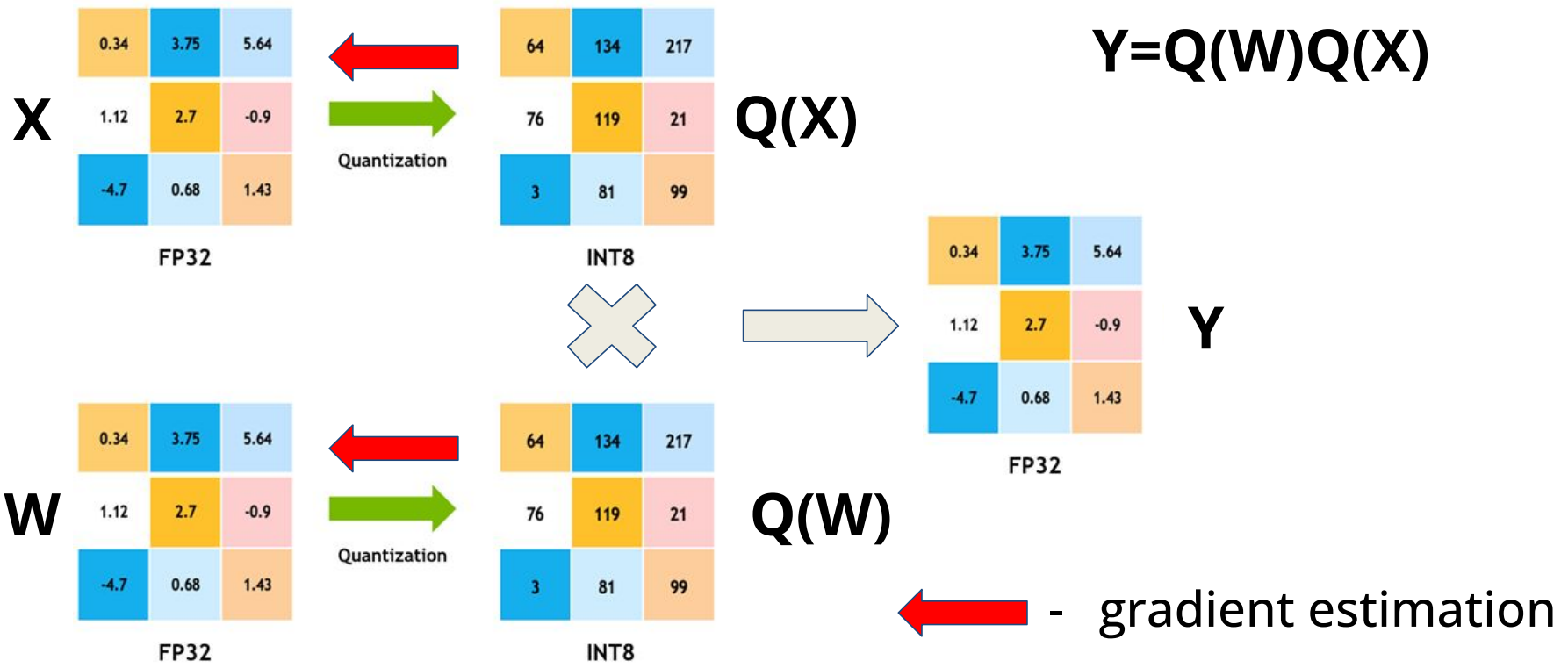
Quantized Forward Pass



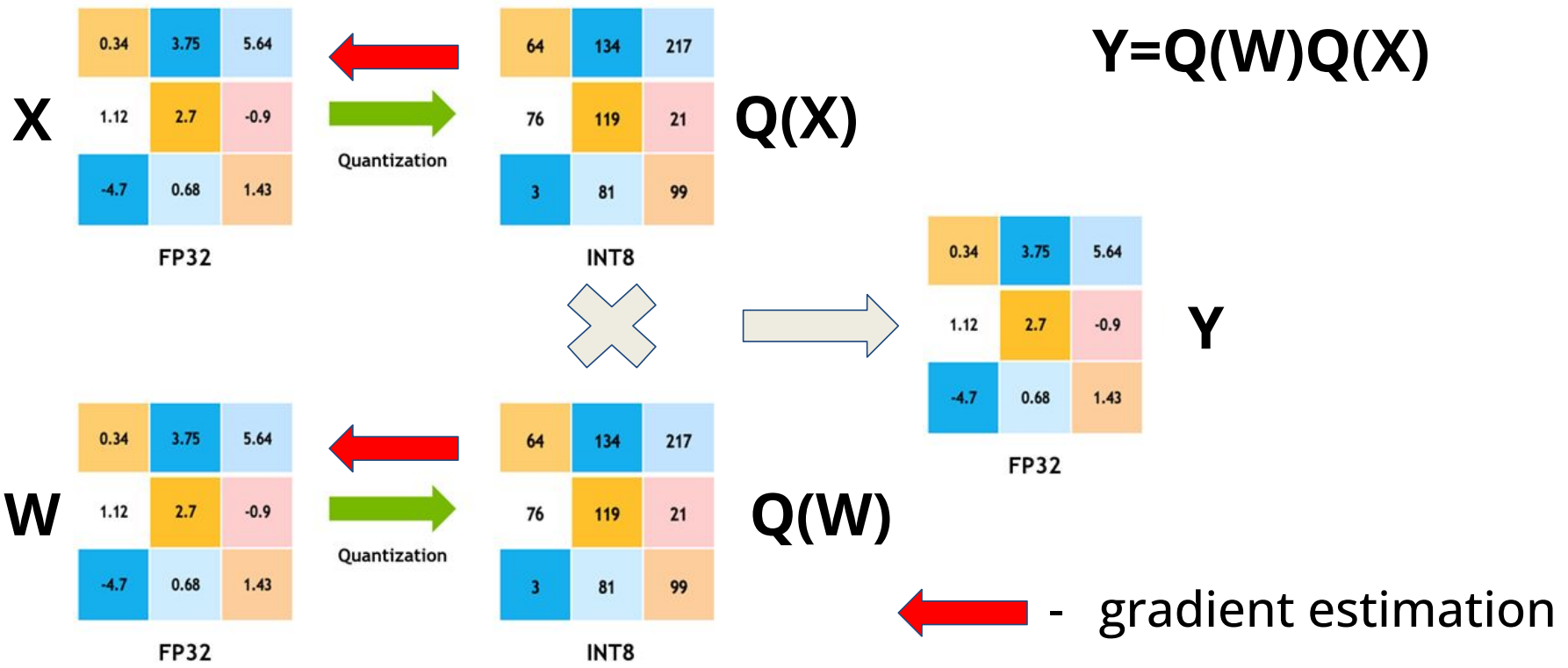
Quantized Forward Pass



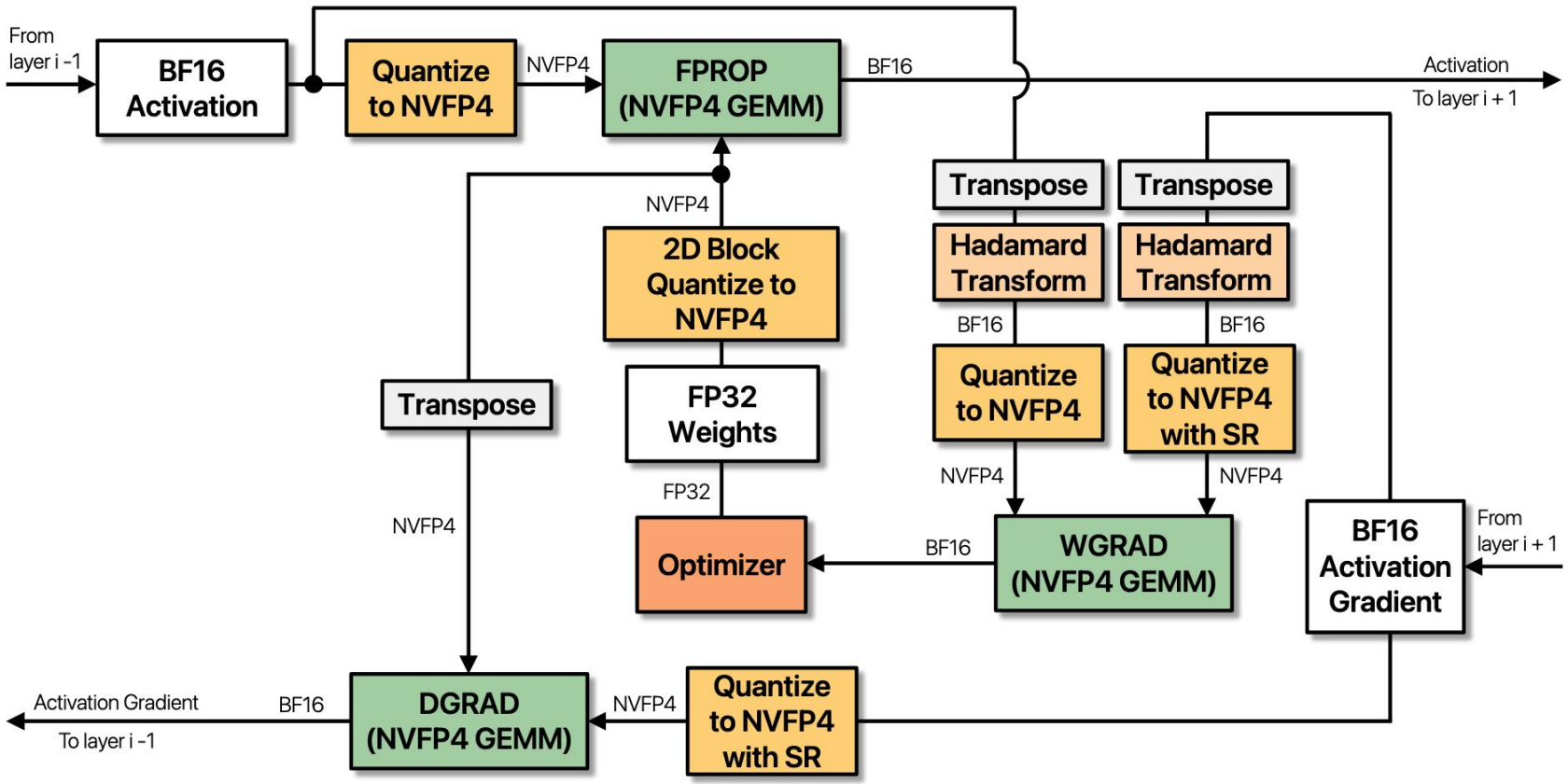
Quantized Forward Pass



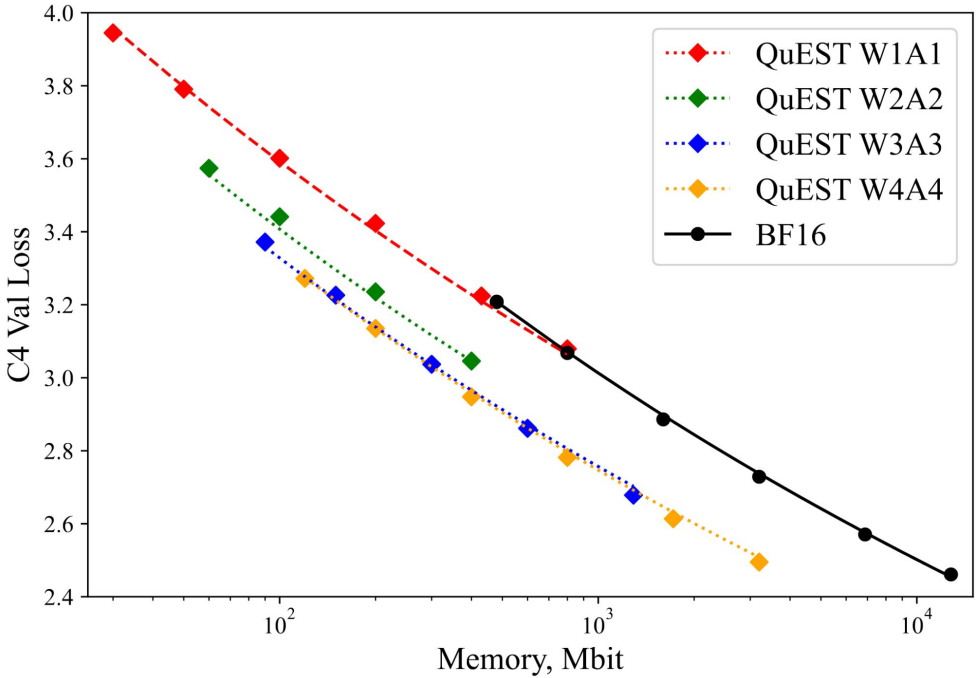
Quantized Forward Pass

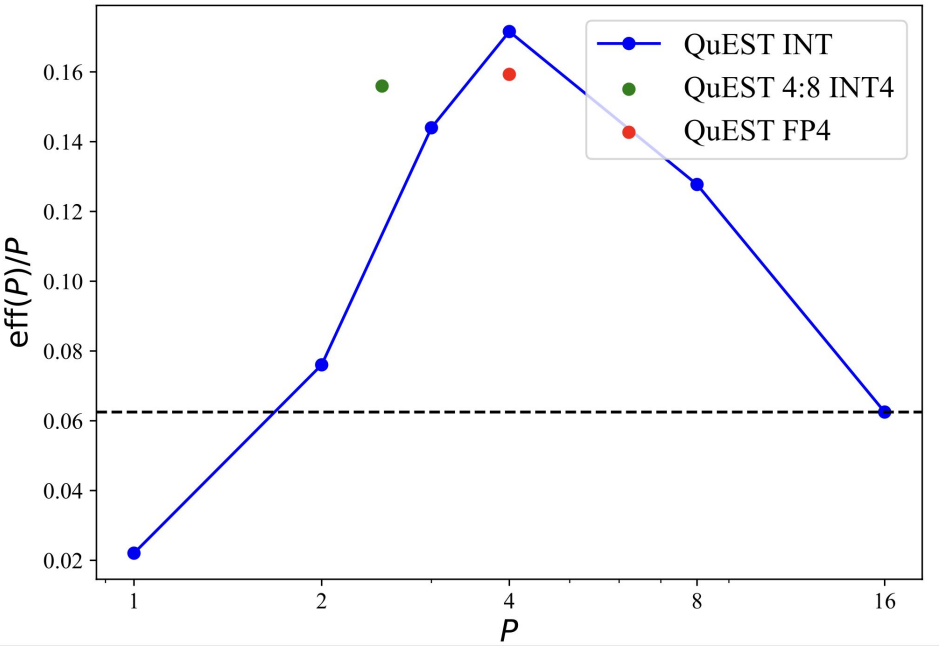


Quantized Everything

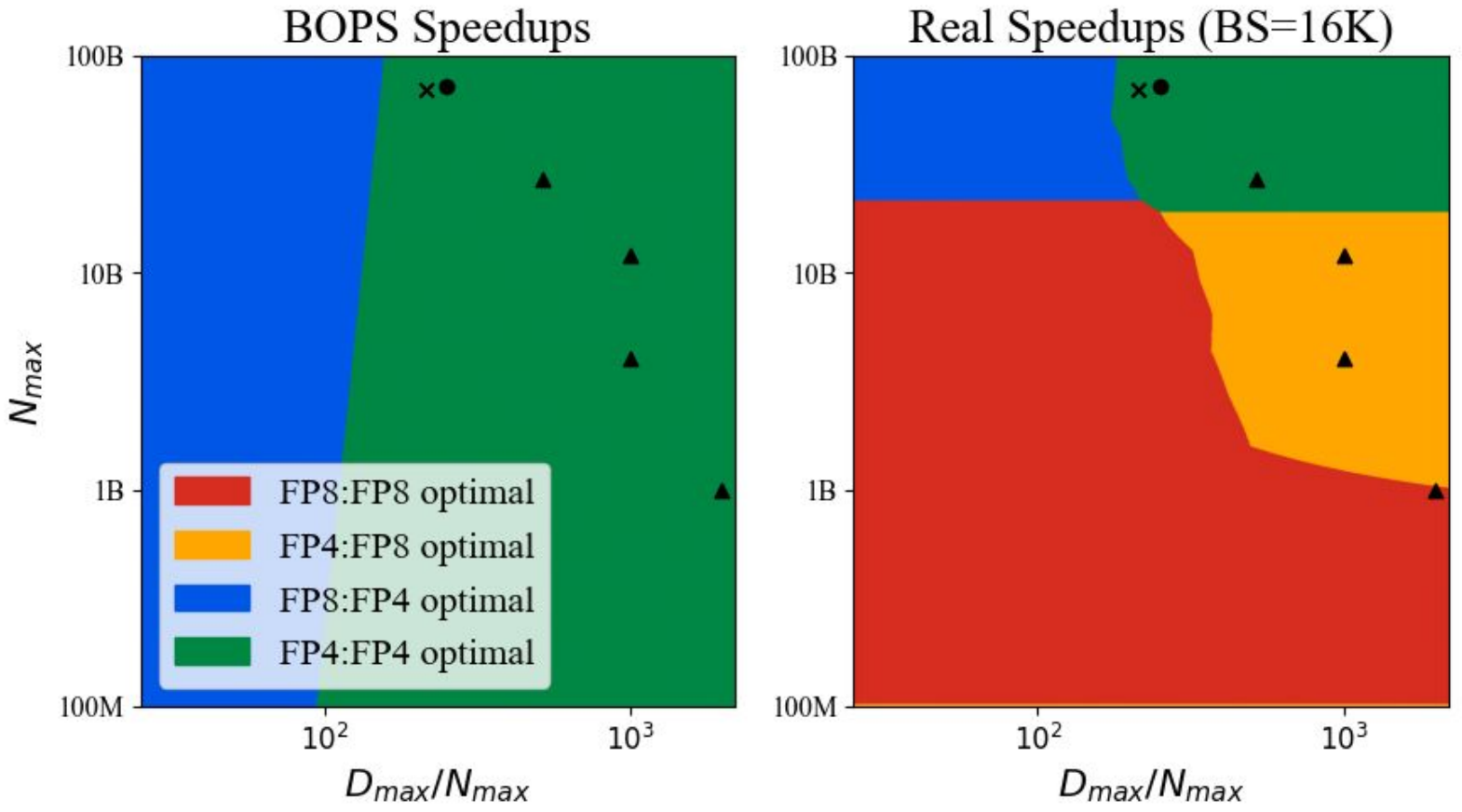


~4 Bit is Pareto-Optimal





Keep Speedups in Mind



More QAT Materials

QuEST (SOTA QAT):

- Paper (ICML 2025)
- YouTube Lecture

Quartet (QuEST for MXFP4):

- Paper (NeurIPS 2025)
- YouTube Lecture

NVFP4 by NVIDIA:

- Report

Homework:

- ***hw_8.ipynb***:
  - 3pt for Speculative Decoding with N-Grams
  - 7pt for GPTQ
  - (Bonus) +3pt for QUIK - fancier GPTQ for W+A
- Subscribe to